

ZORVEN AI

Onboarding Intelligence Agent

User Story Backlog — build-ready slices for onboarding-intelligence-agent-svc

Version 2.0 · 25 July 2026

Zorven AI · Prevision_WS monorepo

Sliced from Design Document v2.0 and Requirements v1.1 · Supersedes Backlog v1.0

0. How to Read This Backlog

A backlog is not a list of intentions. It is the contract between the design document and the person — or the coding agent — who has to make the thing exist. v1.0 of this backlog failed that contract in one specific, measurable way: its acceptance-criteria column was headed *"Acceptance criteria (condensed)"* and held semicolon-joined fragments. A fragment like *"Endpoints per design §9.2 with RBAC (Editor+ create, Viewer read); state transitions validated server-side (illegal transition → 409); OpenAPI docs"* tells you roughly what to build. It does not tell you when you are done, and it cannot be handed to Claude Code.

What changed in v2.0. Every story is now a card rather than a table row. Every card carries:

- **Behavioural acceptance criteria in Given / When / Then form**, numbered **AC-n**, each one independently testable and each one written so that a disagreement about whether the story is done can be settled by running something rather than by discussing it.
- **Technical notes anchored to real file paths** from Design §4.4 — `app/logic/live_session.py`, `app/cache/redis_manager.py`, `config/skills.yaml` — not to hypothetical modules. Where a story touches the existing Django backend or `prompt-optimization-svc`, the real path in that repository is named.
- **Named test cases** mapped to the 21 test files enumerated in Design §22, so the test suite grows story by story instead of arriving as a week-15 surprise.
- **Explicit dependency edges in both directions** — what blocks this, and what this unblocks — so a re-plan does not have to be re-derived from prose.

Identifier scheme. Story IDs are `<epic letter>-<two digits>`: **A-01**, **B-05**, **L-02**. Design v2.0 already cites four of these by name — **A-02** (the Kong WebSocket spike, §23 OD-1), **A-04** (raising Redis `databases` to 28, §4.2 and §14), **B-05** (the `FieldProvenance` check constraint, §10.1) and **L-02** (the `onboarding.golden-dataset.candidates` topic, §13.1) — and this backlog holds those assignments fixed. Do not renumber them.

Estimation. Story points, calibrated for one developer: 1 ≈ half a day, 2 ≈ a day, 3 ≈ two days, 5 ≈ half a week, 8 ≈ a full week. Spikes are timeboxed rather than estimated, and their point value is the box.

1. Definition of Ready

A story may not enter a week's plan until all of the following hold. This exists because the most expensive failure mode for a three-person team is starting a story that cannot finish.

- The design section named in the story header has been read, and any ambiguity in it has been resolved into either an acceptance criterion or an entry in Design §23 (Open Design Decisions).
- All upstream dependencies listed in **Depends on** are **Done** — or an explicit stub has been agreed and written down, with the story that removes the stub already in the backlog.
- Any third-party credential, quota or platform change the story needs (Google STT project, Vision API enablement, Kong route, Redis config) exists in the staging environment. **A story is never Ready if it depends on a platform change that has not landed.**
- The story's test files exist in the repository, even if only as skipped stubs, so that "add the test" is never the thing that gets dropped on a Friday.
- The point estimate has been agreed by at least two of the three team members.

2. Definition of Done

Every story, without exception:

- Code merged after review. All acceptance criteria in the card have a corresponding passing assertion, and the specific test cases named in the card's test table exist and pass.
- Existing suites stay green. **NFR-COMPAT** is not a nice-to-have: the manual, no-meeting onboarding path must still work end to end after every merge, and `tests/e2e/test_process_to_review.py` covers it.
- Tenant isolation respected. No Redis key, no queryset and no GCS path is constructed without a tenant argument; `tests/test_redis_key_isolation.py` enforces this structurally.
- Events emitted where Design §12 requires them, and no event payload carries a value that Design §12 says it must not — enforced by `tests/test_events_no_pii.py`.
- Guardrail and RBAC paths touched by the story have a positive and a negative test case.
- Deployed to Railway staging and demoable at the Friday review.
- **CLAUDE.md** and the service registry updated when the service's external surface changed (new endpoint, new port, new env var, new topic).

The compatibility rule is load-bearing. Every existing Zorven tenant is mid-onboarding under the old flow or has completed it. Every migration in Epic B is additive, every new **Company** field is nullable, and no story is Done if it makes the five-page wizard require a meeting.

3. Epic Map

Epic	Theme	Stories	Points	Design sections
A	Foundations, scaffold and spikes	A-01 ... A-06	16	\$4.1–4.4, \$12, \$23
B	Backend data model and session APIs	B-01 ... B-08	19	\$10.1, \$10.2
C	PREP — research and questionnaire via chat	C-01 ... C-05	14	\$2.1, \$8.1, \$10.2.1
D	Calendar and scheduling	D-01 ... D-03	8	\$10.2, \$11
E	Onboarding Interface shell	E-01 ... E-02	5	\$11
F	Recording and real-time transcription	F-01 ... F-06	18	\$4.3, \$9.2, \$10.2.3
G	Live meeting assist	G-01 ... G-06	17	\$5.1, \$6, \$8.1, \$9.2
H	Document capture and OCR	H-01 ... H-04	11	\$8.4, \$10.1
I	Recordings library	I-01 ... I-03	8	\$8.1 SKL-OIA-08, \$11
J	Processing and auto-fill	J-01 ... J-06	19	\$5.3, \$9.3, \$10.2.2
K	Review, wizard extension and PDF	K-01 ... K-05	12	\$10.3, \$11
L	prompt-optimization-svc integration	L-01 ... L-05	13	\$13.1, \$17
M	Security, GDPR and operations	M-01 ... M-05	13	\$5, \$15, \$16, \$19, \$20
N	End-to-end hardening	N-01 ... N-03	8	\$22
	Total	67 stories	181	

The point total is higher than v1.0's 167 across 58 stories, and that is deliberate rather than accidental scope creep. Three of v1.0's stories were compound — A3 bundled a platform-wide Redis config change with a service scaffold, and B1 bundled six Django models including the one carrying the check constraint — and splitting them exposed work that was previously hidden inside an estimate.

4. Epic A — Foundations, Scaffold and Spikes

Weeks 1 and 2 exist to make the expensive decisions cheap. Two spikes run first because each one can force a topology change, and a topology change discovered in week 6 costs a sprint.

A-01 • Spike — Google STT v2 streaming latency and diarization

3 pts (timeboxed to 2 days) • Engineering team • **Depends on** — • **Blocks** F-05, F-06 • **Design** \$4.3, \$18.3, \$23 OD-2 • **Requirements** FR-REC-03, NFR-PERF-01

As the engineering team, **we want** a Google Cloud Speech-to-Text v2 streaming prototype measured end to end from a real browser microphone, **so that** the ≤ 2 s partial-latency budget in Design \$18.3 is a measured number rather than a hope, and so that the two-speaker diarization assumption underlying SKL-OIA-04 is proven before six stories are built on it.

Acceptance criteria

AC-1 • Latency is measured, not estimated

- **Given** a throwaway relay deployed to Railway staging that forwards browser microphone audio to the STT v2 streaming API
- **When** a two-minute sample of natural two-person conversation is spoken into it
- **Then** the elapsed time from utterance onset to first partial transcript is recorded for at least 100 utterances, and p50 and p95 are written into the spike note
- **And** the p95 figure is compared against the 2 s budget with an explicit pass, marginal or fail verdict

AC-2 • Diarization is inspected on real two-speaker audio

- **Given** the same relay configured with `diarization_config.min_speaker_count = 2` and `max_speaker_count = 2`
- **When** the two-minute two-person sample is processed
- **Then** the per-segment speaker tags are exported and manually compared against ground truth
- **And** the note records the observed misattribution rate and, critically, whether speaker labels remain stable across a mid-stream reconnect

AC-3 • Cost is known before commitment

- **Given** the measured audio duration and the published STT v2 streaming rate for the chosen region
- **When** cost is extrapolated to a 60-minute onboarding meeting
- **Then** a per-meeting-hour figure appears in the note, together with the incremental cost of enabling diarization and of the batch backfill path used by F-06

AC-4 • The language decision is closed

- **Given** OD-2 asks whether language is fixed per tenant or auto-detected
- **When** the spike completes
- **Then** the note recommends one option with evidence, and OD-2 is either resolved in the design or explicitly re-scheduled with a new "needed by" date

Technical notes

- The spike code is throwaway and lives on a branch; it must not be merged into `app/providers/stt.py`. What is merged from this spike is the *measurement* and the decision.

- Use the v2 `StreamingRecognize` API with `interim_results = true`. Note that v2 requires a recognizer resource per region — creating it is part of the spike and its name becomes `OIA_STT_RECOGNIZER` in Design §19.
- Measure from utterance onset in the browser (`AudioWorklet` timestamp at first sample above the noise floor) rather than from the relay's receive time. Measuring at the relay hides browser buffering, which is a real component of what the operator perceives.
- Record the audio fixture used and check it into `tests/fixtures/` — F-05 and `tests/test_stt_adapter.py` will need exactly this file.

Test cases

File	Case	Proves
—	Spike; no production tests	The deliverable is a written note plus a checked-in audio fixture, not code
<code>tests/fixtures/two_speaker_2min.wav</code>	Fixture committed	F-05 and <code>test_stt_adapter.py</code> have a real input to work against on day one

Risk. If p95 partial latency exceeds ~3 s, the live-assist experience changes character: green signals arrive after the operator has already moved on. The fallback is not to abandon live assist but to shift SKL-OIA-05 to fire on explicit checkbox interaction rather than continuously. Decide this in the spike, not in week 7.

A-02 • Spike — Kong WebSocket upgrade on Railway

3 pts (timeboxed to 2 days) • Engineering team • **Depends on** — • **Blocks** F-04 • **Design** §4.1, §10.2.3, §23 OD-1 • **Requirements** FR-LIVE-01, NFR-SEC-02

As the engineering team, **we want** to prove that Kong on Railway proxies an authenticated WebSocket upgrade to a FastAPI service and holds the connection open under realistic conditions, **so that** LIVE mode's single riskiest infrastructure assumption is settled in week 1 rather than discovered in week 6 when six stories already depend on it.

Acceptance criteria

AC-1 • The upgrade completes through the gateway

- **Given** a Kong route `/api/v1/agents/onboarding/live` on Railway staging pointing at a throwaway echo service
- **When** a browser opens `wss://<staging-gateway>/api/v1/agents/onboarding/live/test-session` carrying a valid tenant JWT
- **Then** the socket reaches `OPEN` and a 1 KB binary frame round-trips
- **And** the same request with an invalid or absent JWT is rejected by Kong before reaching the service, closing with 4401

AC-2 • The connection survives a realistic meeting

- **Given** an open socket through Kong
- **When** audio-sized frames are sent continuously for 45 minutes with a 20 s application heartbeat

- **Then** the connection is not closed by Kong, Railway's edge, or any idle timeout
- **And** if any timeout is observed, its source and duration are identified and the required keepalive interval is recorded

AC-3 • Reconnect behaviour is characterised

- **Given** an open socket carrying a monotonic `seq` on every frame
- **When** the network is interrupted for 10 s and the client reconnects with the last `seq` it saw
- **Then** the observed behaviour — whether Kong preserves anything, how long re-establishment takes, whether the JWT is re-validated — is documented in the note
- **And** the note states whether the zero-token replay design in Design §9.2 is implementable as specified

AC-4 • A go / no-go decision is recorded

- **Given** all measurements above
- **When** the spike closes
- **Then** OD-1 is resolved in the design document with one of: proceed through Kong; proceed through Kong with a named configuration change; or bypass Kong via a direct service route with an explicitly documented auth compensation

Technical notes

- Kong needs no special WebSocket plugin, but the route's `protocols` must include `https` and the upstream must not have `buffering` enabled. Check the existing gateway config in the monorepo's Kong declarative file before assuming defaults.
- Railway's edge proxy is the more likely source of an idle timeout than Kong itself. Test both by hitting the service's direct Railway URL and the Kong URL, and compare — this isolates which layer imposes the limit.
- JWT propagation matters: confirm the tenant claim survives Kong's transformation and arrives at the service in the header `app/api/deps.py` expects. A socket that opens but arrives tenant-less is a failure of this spike, not a later story's problem.
- If a bypass is chosen, IG-05 tenant validation moves entirely into `app/api/ws.py` and its cost must be added to F-04.

Test cases

File	Case	Proves
—	Spike; no production tests	Deliverable is a go/no-go note plus a resolved OD-1
<code>tests/test_ws_handshake.py</code>	Stub committed, skipped	F-04 inherits a file rather than creating one

Risk. This is the story most likely to change the architecture. If Kong cannot hold a 45-minute socket, the choice is a direct route (losing gateway-level rate limiting, which then has to be rebuilt in `app/api/ws.py`) or chunked HTTP polling (losing the ≤ 2 s partial budget outright). Both are survivable; neither is cheap in week 6.

A-03 • Raise the Kafka and observability baseline for the new service

2 pts • Developer • **Depends on** A-05 • **Blocks** every story that emits an event • **Design** §12, §13.1, §20 • **Requirements** NFR-OPS-01

As a developer, I want the six fleet-standard Kafka topics provisioned for `onboarding-intelligence-agent-svc` and the structured event emitter wired with OpenTelemetry, so that every subsequent story can emit auditable events without each one re-solving plumbing.

Acceptance criteria

AC-1 • Topics exist and are reachable from the service

- **Given** the Terraform module that provisions fleet agent topics
- **When** it is applied with the `onboarding-intelligence-agent` name variable
- **Then** `agent.commands.onboarding-intelligence-agent`, `agent.results.onboarding-intelligence-agent`, `agent.events.<tenant>`, `agent.escalations`, `agent.dlq.onboarding-intelligence-agent` and the heartbeat topic exist on staging
- **And** the service publishes and consumes a round-trip message on each at startup in a smoke check

AC-2 • The event envelope is the one in the design

- **Given** the `AgentEvent` Pydantic envelope specified in Design §12
- **When** any code path calls `EventEmitter.emit(...)`
- **Then** the emitted payload validates against that model, carrying `event_id`, `event_type`, `tenant_id`, `session_id`, `trace_id`, `span_id`, `occurred_at` and `payload`
- **And** a payload that fails validation raises rather than silently emitting a malformed event

AC-3 • Traces correlate across the service boundary

- **Given** a request arriving from Django carrying a W3C `traceparent`
- **When** the service handles it and emits events
- **Then** the emitted `trace_id` matches the incoming trace, and the span appears in the same trace tree as the Django span in the collector

Technical notes

- `app/events/catalog.py` holds the `EventType` enum. Populate it with the full `EVT-001...012` and `EVT-101...110` set from Design §12 now, even though most are unused until later epics — a complete enum makes the later stories one-line additions and makes `tests/test_events_no_pii.py` meaningful from week 2.
- `app/core/telemetry.py` mirrors `voc-agent-svc`'s setup; the only new work is the WebSocket span convention, where one span covers the whole live session and child spans cover each analysis batch. Long-lived spans need `span.set_attribute` updates rather than nesting, or the trace becomes unreadable.
- Do not create `onboarding.golden-dataset.candidates` here. That topic is L-02's, and creating it early produces an empty topic nobody consumes.

Test cases

File	Case	Proves
<code>tests/test_kafka_roundtrip.py</code>	<code>test_publish_consume_each_fleet_topic</code>	Every §13.1 fleet topic accepts and returns a message
<code>tests/test_events_no_pii.py</code>	<code>test_envelope_validates</code>	The <code>AgentEvent</code> model rejects a payload missing <code>trace_id</code> or <code>tenant_id</code>
<code>tests/test_events_no_pii.py</code>	<code>test_event_type_enum_complete</code>	All 22 event types from §12 are present in the enum

A-04 · Raise Redis `databases` to 28 across the shared instance

2 pts · Developer · Depends on — · Blocks A-05, and transitively every story touching Redis · Design §4.2, §14 · Requirements NFR-OPS-02

As a platform developer, I **want** the shared Redis instance reconfigured from `databases 27` to `databases 28` and the change verified against every service that shares it, **so that** DB 27 exists before any OIA code tries to select it.

This story is separate from the service scaffold on purpose. `redis.conf` currently declares `databases 27`, which yields valid indices 0 through 26. Databases 0–26 are already allocated across the fleet. **SELECT 27 does not fail gracefully — it returns `ERR DB index is out of range`**, and it does so at connection time, meaning the service will not start. This is a platform-wide configuration change requiring a restart of an instance that every other agent depends on, so it is scheduled in week 1 with its own review and its own rollback plan.

Acceptance criteria

AC-1 · The configuration change is applied and verified

- **Given** the shared Redis instance with `databases 27`
- **When** `redis.conf` is updated to `databases 28` and the instance is restarted in the staging environment
- **Then** `CONFIG GET databases` returns 28, and `SELECT 27` succeeds from a plain `redis-cli` session

AC-2 · No existing service is disturbed

- **Given** the fleet services currently using DBs 0–26
- **When** the restarted instance comes back
- **Then** every service's `/health` endpoint reports its Redis dependency healthy within 60 s
- **And** no key count on any existing DB has changed, verified by capturing `DBSIZE` for 0–26 before and after

AC-3 · The change is replayed to production with a rollback plan

- **Given** a successful staging application
- **When** the change is scheduled for production
- **Then** a written rollback step exists (revert to 27; no OIA service deployed yet, so nothing depends on 27 in production until A-05 ships)
- **And** the production change is applied in a window agreed with the team, ahead of the first production OIA deploy

Technical notes

- If Redis is managed (Railway plugin, Redis Cloud) rather than self-hosted from a `redis.conf`, `databases` may not be operator-settable. **Confirm this on day one of week 1.** If it is not settable, the fallback is a key-prefix namespace on an existing DB rather than a dedicated DB — which changes every key pattern in Design §14 and makes `tests/test_redis_key_isolation.py` more important, not less. Escalate immediately; do not silently pick DB 26.
- Persistence: if the instance uses RDB snapshots, take one before the restart. Adding a database index does not migrate data, but a restart is a restart.
- Update the monorepo `CLAUDE.md` Redis allocation table in the same PR so the next service does not repeat this discovery.

Test cases

File	Case	Proves
tests/ test_redis_key_isolation.py	test_db_27_selectable	The service's configured DB index is reachable; fails loudly if the config change was missed in an environment
Manual runbook check	DBSIZE diff for DBs 0-26	The restart cost the fleet nothing

A-05 • Scaffold `onboarding-intelligence-agent-svc`

3 pts • Developer • **Depends on** A-04 • **Blocks** A-06, C-01, L-01 • **Design** §4.2, §4.4, §19 • **Requirements** NFR-OPS-01

As a developer, I **want** the service scaffolded on port 8120 with the `OIA_` settings prefix, the Design §4.4 module layout, and a health probe that actually checks its dependencies, **so that** every later story has a home and a deploy target.

Acceptance criteria**AC-1 • The service matches the fleet layout**

- **Given** Design §4.4
- **When** the repository is created
- **Then** every directory and module in that tree exists, with `NotImplementedError` bodies where the story that fills them has not run yet
- **And** `pyproject.toml` pins Python 3.12, Black at 88 characters, `mypy --strict` and `pytest-asyncio` in `asyncio_mode = "auto"`, matching `voc-agent-svc`

AC-2 • Configuration is typed and prefixed

- **Given** `app/core/config.py` defining `Settings(BaseSettings)` with `env_prefix = "OIA_"`
- **When** the service starts without a required variable set
- **Then** it fails at startup with a Pydantic validation error naming the missing variable, rather than starting and failing on first use
- **And** all thirteen settings from Design §19 are declared, with the documented defaults

AC-3 • The health probe is honest

- **Given** the service running
- **When** `GET /health` is called
- **Then** it returns 200 only when Redis DB 27 responds to `PING` and the Kafka producer reports a live broker connection
- **And** `GET /health/diagnostics` returns per-dependency status including Redis, Kafka, the Django backend, POI and the configured GCS bucket
- **And** with Redis stopped, `/health` returns 503 within 2 s rather than hanging

AC-4 • It is deployed and registered

- **Given** `Dockerfile` and `railway.json`
- **When** the service is deployed to Railway staging

- **Then** it is reachable on its assigned port, `/health` is green
- **And** the monorepo `CLAUDE.md` service registry records port 8120, env prefix `OIA_`, Redis DB 27, and the shared DB 2 prompt cache

Technical notes

- Copy `voc-agent-svc`'s `Dockerfile` and `railway.json` verbatim and change only the service name and port. Divergence in build tooling across the fleet is a slow tax.
- `app/core/config.py` should read the GCS bucket, Kafka bootstrap and Redis URL from environment references rather than literals — Design §19 is explicit that secrets are never inline.
- The `/health` Redis check must `SELECT 27` explicitly, not merely `PING` on DB 0. A `PING` on the default DB will pass on an instance where A-04 was never applied, which defeats the point.
- `app/api/ws.py`, `app/logic/live_session.py`, `app/cache/session_store.py`, `app/cache/idempotency.py`, `app/core/errors.py` and `app/providers/` are all marked NEW in Design §4.4. Create them empty here; they have no counterpart to copy from.

Test cases

File	Case	Proves
<code>tests/test_skills_yaml.py</code>	<code>test_file_parsing</code>	<code>config/skills.yaml</code> exists and parses, even while empty of skills
Smoke	<code>GET /health</code> with Redis down returns 503	The probe checks rather than reports optimism
Smoke	Missing <code>OIA_GCS_BUCKET</code> fails startup	Configuration errors surface at deploy, not at first meeting

A-06 • Agent skeleton — registry, guardrail chain, RBAC evaluator

3 pts • Developer • Depends on A-05 • Blocks every skill story • Design §4.5, §5, §8, §15 • Requirements NFR-SEC-01

As a developer, I want `SkillRegistry`, the ordered guardrail chain, the RBAC evaluator and the `BaseSkill` / `StreamingSkill` interfaces implemented against `config/skills.yaml`, so that all sixteen skills plug in uniformly and no skill can bypass a guardrail by being written carelessly.

Acceptance criteria

AC-1 • Skills load from configuration, not from imports

- **Given** `config/skills.yaml` containing the sixteen skill declarations from Design §8
- **When** the service starts
- **Then** `SkillRegistry` loads every declaration, resolves each to its implementing class, and exposes lookup by both `skill_id` and `name`
- **And** a declaration whose implementing class is missing fails at startup with a message naming the skill, rather than at first invocation

AC-2 • Guardrails run in a fixed order and cannot be skipped

- **Given** a skill invocation
- **When** it executes through the registry

- **Then** input guardrails (IG) run before the prompt is built, processing guardrails (PG) run around execution, and output guardrails (OG) run before the result is returned
- **And** each guardrail evaluation emits a structured log line carrying rule id, verdict and elapsed time
- **And** a skill invoked directly, bypassing the registry, is impossible — `BaseSkill.__call__` is not the public entry point and the registry's `execute` is

AC-3 • RBAC denies produce a typed error and an event

- **Given** the §15 role-to-capability matrix
- **When** a VIEWER invokes a skill whose `allowed_roles` excludes VIEWER
- **Then** the call raises the typed `ERR-03` authorization error, returns HTTP 403 with the standard error body from Design §18.4
- **And** an `rbac.violation` event is emitted carrying role, skill id and tenant, and carrying no request payload

AC-4 • The YAML contract is enforced by test

- **Given** `tests/test_skills_yaml.py` adapted from the fleet original
- **When** it runs
- **Then** it asserts `EXPECTED_SKILL_COUNT = 16`, `EXPECTED_ID_PREFIX = "SKL-0IA-"`, IDs matching `^SKL-0IA-\d{2}[a-z]?$`, roles drawn only from `{OWNER, ADMIN, EDITOR, VIEWER}`, `timeout_ms ≤ 120000`, and every `input_schema` entry carrying `field`, `type` and `required`

Technical notes

- `app/skills/models.py` carries `SkillMeta` in the fleet shape: `skill_id`, `name`, `description`, `allowed_roles` defaulting to all four, `idempotent = True`, `max_retries = 1`, `timeout_ms = 30000`, `circuit_breaker_dependency = ""`. Do not add fields; a diverging dataclass breaks the shared contract test.
- `StreamingSkill` is the one genuine extension over `voc-agent-svc`. It returns an async iterator rather than a `SkillResult`, which means output guardrails must run per-yield rather than once. Implement OG as an async generator wrapper so the same rule objects serve both interfaces.
- The guardrail chain lives in `app/logic/guardrails.py`. Implement the chain and the registration mechanism here with rules as no-op stubs; M-01 fills in every rule body. This story proves the *ordering*, not the rules.
- The RBAC matrix belongs in `app/rbac/engine.py` as data, parameterised the same way `tests/test_rbac.py` will parameterise it. A matrix written as `if` statements cannot be exhaustively tested.

Test cases

File	Case	Proves
<code>tests/test_skills_yaml.py</code>	Full contract suite	Sixteen skills, correct ID pattern, valid roles, timeout ceiling, schema keys
<code>tests/test_rbac.py</code>	<code>test_matrix_exhaustive</code>	Every role against every capability, allow and deny both asserted
<code>tests/test_guardrails.py</code>	<code>test_hook_ordering</code>	IG before prompt build, PG around execution, OG before return — asserted by recording call order

File	Case	Proves
tests/test_guardrails.py	test_streaming_og_per_yield	Output guardrails evaluate each yielded chunk, not only the final one

5. Epic B — Backend Data Model and Session APIs

Every model in this epic lives in a new `apps/onboarding/` Django app inside `ai-brand-automator`. Every migration is additive and reversible, and no existing table is altered destructively. The epic is split more finely than v1.0 split it, because v1.0's single B1 bundled six models — including the one carrying the constraint that makes the whole provenance design real — into one five-point estimate.

B-01 • Core session models and the `apps/onboarding` app`

3 pts • Developer • Depends on — • Blocks B-02, B-04, B-05 • Design §10.1, §9.4 • Requirements FR-PREP-01, NFR-COMPAT

As a developer, I want `OnboardingSession`, `Questionnaire` and `Question` created in a new `apps/onboarding/` Django app with additive migrations, so that the prepare-and-meet flow has a schema before any API or agent code needs one.

Acceptance criteria

AC-1 • The three models match the design exactly

- **Given** Design §10.1
- **When** the migration is applied
- **Then** `OnboardingSession` carries `tenant` FK, `company` FK, `status`, `escalated_from`, `questionnaire` FK (nullable), `created_by`, `prompt_versions` JSON, `evidence_manifest_hash` and timestamps
- **And** `Questionnaire` carries `tenant`, `company`, `session` FK, `status`, `depth`, `question_count`, `source_chat_session_id`, `approved_by`, `approved_at`, `version` and `is_template`
- **And** `Question` carries `questionnaire` FK, `order`, `text`, `origin`, `workflow_target`, `target_field`, `status`, `sufficiency_score`, `answer_summary` and `evidence` JSON

AC-2 • One live session per company is a database property

- **Given** a company with an `OnboardingSession` in a non-terminal status
- **When** a second non-terminal session is created for the same company
- **Then** the insert fails on a partial unique index, not on application logic
- **And** creating a second session succeeds once the first reaches `COMPLETED` or `ARCHIVED`

AC-3 • Existing tenants are untouched

- **Given** the full existing test suite for `ai-brand-automator`
- **When** the migration is applied and the suite runs
- **Then** it is green, and no query plan for an existing endpoint has changed
- **And** the migration reverses cleanly on a database containing rows

AC-4 • The models are operable

- **Given** Django admin
- **When** an Owner opens it
- **Then** all three models are registered with tenant-scoped querysets and readable list displays
- **And** model factories exist in `apps/onboarding/tests/factories.py` for use by every later story

Technical notes

- Register the app in `INSTALLED_APPS` and wire its queriesets through the existing `JWTTenantMiddleware` tenant scoping — a manager that does not filter by tenant is the single most likely source of a cross-tenant leak, and `RoleBasedPermissionMixin` will not save you at the ORM layer.
- `sufficiency_score` and `evidence` on `Question` must be written together or not at all (OG-06). Enforce it in the model's `save()` as well as in the serializer; the agent is not the only writer.
- `prompt_versions` is a JSON dict of `{prompt_name: version}` pinned at session start. Design §17.2 makes this the mechanism that prevents a mid-meeting POI promotion from changing behaviour. It is written once, at session creation, by L-03.
- Choose `status` choices verbatim from the state diagram in Design §9.4 — `DRAFT`, `PREPARING`, `READY`, `MEETING_LIVE`, `GATHERED`, `PROCESSING`, `REVIEW_PENDING`, `CONFIRMED`, `COMPLETED`, `ESCALATED`. Do not invent intermediate values; B-04 validates transitions against exactly this set.

Test cases

File	Case	Proves
<code>apps/onboarding/tests/test_models.py</code>	<code>test_one_active_session_per_company</code>	Second non-terminal session raises <code>IntegrityError</code>
<code>apps/onboarding/tests/test_models.py</code>	<code>test_question_score_and_evidence_atomic</code>	Saving a score without evidence raises
<code>apps/onboarding/tests/test_migrations.py</code>	<code>test_migration_reversible</code>	Forward then backward on a populated database leaves no residue
Existing suite	Full run	NFR-COMPAT holds

B-02 • Meeting evidence models and `BrandAsset` extension

3 pts • Developer • Depends on B-01 • Blocks B-07, B-08, F-02, H-01 • Design §10.1, §24 • Requirements FR-REC-01, FR-CAP-01, NFR-PRIV-02

As a developer, I want `MeetingRecording` and `ConsentRecord` created and `BrandAsset` extended with `usage_tag`, `onboarding_session`, `ocr_text` and `ocr_confidence`, so that recordings, consent and captured media all have somewhere to live before the recording epic starts.

Acceptance criteria

AC-1 • Recording rows model one start/stop cycle each

- **Given** `MeetingRecording` with `session` FK, `modality`, `audio_asset` FK to `BrandAsset`, `transcript_gcs_path`, `duration_s`, `status`, `summary` JSON, `started_at` and `stopped_at`
- **When** an operator starts and stops recording three times in one meeting
- **Then** three rows exist, each with its own duration and status, all pointing at the same session

AC-2 • `modality` is present and defaulted

- **Given** Design §24's commitment that adding video in v2 is a data-free change
- **When** a recording is created without specifying modality
- **Then** it is `AUDIO`
- **And** `VIDEO` is a declared, unused choice — present in the enum, rejected by no constraint

AC-3 • Consent is a record, not a boolean

- **Given** `ConsentRecord` with `session` FK, `subject_name`, `granted_by`, `method`, `scope` JSON, `granted_at` and nullable `revoked_at`
- **When** consent is recorded
- **Then** all of subject, method, scope and grantor are persisted
- **And** setting `revoked_at` is visible to any consumer querying the session's consent state

AC-4 • `BrandAsset` gains onboarding fields without disturbing existing assets

- **Given** existing `BrandAsset` rows across tenants
- **When** the additive migration runs
- **Then** every existing row has `usage_tag` null, `onboarding_session` null, `ocr_text` null, `ocr_confidence` null
- **And** `usage_tag` choices are exactly `business_photo`, `previous_ad`, `identity_document`, `brand_asset`, `other`
- **And** the existing asset upload flow continues to work with none of the new fields supplied

Technical notes

- `ocr_text` stores *redacted* text only. The unredacted OCR output never reaches the database — Design §5.2 PG-08 requires redaction before persistence, and H-03 implements it. Document this on the field with a `help_text` so a future developer does not "helpfully" store the raw text.
- `usage_tag` and `ocr_text` are carried into RAG document metadata (Design §10.1) so that WF3 can retrieve prior ads by intent. Check the existing RAG sync payload builder in the ingestion pipeline and extend it in this story, or the fields are orphaned until someone notices in WF3.
- `audio_asset` being a FK to `BrandAsset` rather than a raw GCS path is deliberate: it means recordings inherit the existing landing-bucket, Kafka and RAG pipeline for free rather than growing a parallel one.

Test cases

File	Case	Proves
apps/onboarding/tests/test_models.py	test_multiple_recordings_per_session	Three cycles produce three rows
apps/onboarding/tests/test_models.py	test_brandasset_backfill_nullable	Existing rows survive with nulls
apps/onboarding/tests/test_rag_metadata.py	test_usage_tag_in_rag_payload	<code>usage_tag</code> and <code>ocr_text</code> reach the RAG document metadata

B-03 • Extend `Company` with the approved onboarding fields

2 pts • Developer • Depends on B-01 • Blocks J-02, K-03, K-05 • Design §10.1 • Requirements FR-PROC-03, NFR-COMPAT

As a developer, I want `Company` extended with the thirteen approved fields and its serializers updated, so that extraction in Epic J has somewhere to land.

Acceptance criteria

AC-1 • All thirteen fields exist and are optional

- **Given** Design §10.1's `Company` extension list

- **When** the migration runs
- **Then** `competitors`, `products_services`, `marketing_budget_range`, `digital_presence`, `business_goals`, `founder_story`, `brand_asset_status`, `legal_name`, `trademark_status`, `customer_proof`, `sales_channels`, `audience_languages` and `decision_maker` all exist
- **And** every one is nullable and optional at both the model and serializer layer

AC-2 • Serializers round-trip the new fields

- **Given** the existing `Company` serializer
- **When** a create, update or read is performed
- **Then** the new fields are accepted, persisted and returned
- **And** a payload omitting all of them still succeeds, exactly as before

AC-3 • The wizard and PDF are deliberately not changed yet

- **Given** that K-03 extends the wizard forms and K-05 extends the PDF
- **When** this story merges
- **Then** the wizard renders unchanged and the onboarding PDF field list is unchanged
- **And** this is asserted by a snapshot test, so the decoupling is visible rather than assumed

Technical notes

- JSON-typed fields (`competitors`, `products_services`, `digital_presence`, `customer_proof`, `sales_channels`, `audience_languages`) need a declared shape even though the column is schemaless. Write the expected shape into the serializer as a nested serializer or a JSON schema validator now — J-02's extraction output has to match something, and "whatever the LLM emitted" is not a contract.
- `marketing_budget_range` should be a choice field rather than free text, or every downstream comparison becomes string parsing. Agree the bands with the team before implementing.
- This story deliberately splits from K-03. Landing the schema early lets Epic J proceed while frontend work queues.

Test cases

File	Case	Proves
<code>apps/companies/tests/test_serializers.py</code>	<code>test_new_fields_roundtrip</code>	Create, update and read all carry the new fields
<code>apps/companies/tests/test_serializers.py</code>	<code>test_payload_without_new_fields</code>	Existing clients are unaffected
<code>apps/onboarding/tests/test_pdf_snapshot.py</code>	<code>test_pdf_unchanged_by_b03</code>	The PDF field list has not moved yet

B-04 • Session CRUD and the state machine API

3 pts • Developer • **Depends on** B-01 • **Blocks** E-01, J-01 • **Design** §9.4, §10.2 • **Requirements** FR-PREP-01, NFR-SEC-01

As an Admin, I want session create, read, patch and list endpoints that validate state transitions server-side, **so that** the Onboarding Interface can drive a session without being trusted to know the rules.

Acceptance criteria

AC-1 • Endpoints exist with the documented shapes

- **Given** Design §10.2
- **When** the endpoints are called
- **Then** `POST /api/v1/onboarding/sessions`, `GET /.../{id}`, `PATCH /.../{id}`, `GET /...?company=` all behave per the contract and appear in the OpenAPI schema

AC-2 • Illegal transitions are refused with 409

- **Given** a session in `READY`
- **When** a client PATCHes `status` to `CONFIRMED`, skipping the intervening states in Design §9.4
- **Then** the response is 409 with `ERR-11` and a body naming the current state and the legal next states
- **And** every legal transition in §9.4 succeeds, asserted exhaustively rather than by sampling

AC-3 • Roles are enforced per the matrix

- **Given** the §15 RBAC matrix
- **When** each role calls each endpoint
- **Then** Owner, Admin and Editor may create and patch; Viewer may read only and receives 403 on write
- **And** every response is tenant-scoped, with a cross-tenant session id returning 404 rather than 403

AC-4 • Escalation round-trips

- **Given** a session moved to `ESCALATED` from `PROCESSING`
- **When** it is later resumed
- **Then** `escalated_from` records `PROCESSING` and the session returns to that state, not to the start

Technical notes

- Implement the transition table as data in `apps/onboarding/state.py` — a dict of `{from_state: {allowed_to_states}}` — and have both the model and the serializer consult it. Written as `if` branches, it cannot be exhaustively tested and it will drift from Design §9.4 within two sprints.
- Returning 404 rather than 403 for a cross-tenant id is a deliberate information-disclosure choice consistent with the rest of the platform. Confirm against the existing `RoleBasedPermissionMixin` behaviour and match it.
- The `PATCH` endpoint must not accept `prompt_versions` from a client. That field is written server-side by L-03 only.

Test cases

File	Case	Proves
<code>apps/onboarding/tests/test_session_api.py</code>	<code>test_all_legal_transitions</code>	Parameterised over the §9.4 table
<code>apps/onboarding/tests/test_session_api.py</code>	<code>test_illegal_transition_409</code>	Refusal carries <code>ERR-11</code> and the legal set
<code>apps/onboarding/tests/test_session_api.py</code>	<code>test_cross_tenant_returns_404</code>	No existence disclosure
<code>tests/test_session_state.py</code>	<code>test_escalated_from_roundtrip</code>	Escalation returns to the right state

B-05 • `FieldProvenance` with the grounding check constraint

3 pts • Developer • Depends on B-01, B-02 • Blocks B-06, J-03 • Design §10.1, §5.3 OG-01 • Requirements FR-REV-01, FR-PROC-04

As a developer, I want `FieldProvenance` created with a database-level check constraint forbidding a row whose three source fields are all null, so that "no extracted field without evidence" is a property of the database rather than a property of the agent behaving well.

This is the story Design §10.1 singles out. OG-01 drops ungrounded values in the agent, and that is the right place for it — but the agent is not the only writer. A migration, a data fix, a future refactor or a bug can all insert a row. The constraint makes those writes fail.

Acceptance criteria

AC-1 • The model matches the design

- **Given** Design §10.1
- **When** the migration runs
- **Then** `FieldProvenance` carries `session` FK, `model_name`, `field_name`, `extracted_value`, `final_value`, `classification`, `confidence`, `source_recording` FK null, `source_span` JSON null, `source_media` FK null, `status`, `reviewed_by` and `reviewed_at`
- **And** `(session, model_name, field_name)` is unique

AC-2 • The check constraint rejects ungrounded rows at the database

- **Given** a `FieldProvenance` row with `source_recording`, `source_span` and `source_media` all null
- **When** the row is saved — **by any means**, including `Model.objects.create()`, `bulk_create()`, a raw SQL `INSERT`, and a data migration
- **Then** the write raises `IntegrityError` in every one of those four cases
- **And** the same row with any single source field populated saves successfully

AC-3 • The constraint survives migration replay

- **Given** a database with existing provenance rows
- **When** the migration is reversed and re-applied
- **Then** the constraint is dropped and recreated without data loss, and no existing row violates it

AC-4 • Status transitions are constrained

- **Given** a row in `CONFIRMED` or `EDITED`
- **When** an extraction run attempts to overwrite `extracted_value`
- **Then** the write is refused and a `CONFLICT` row state is recorded instead — the model-level expression of PG-06

Technical notes

- Use Django's `models.CheckConstraint` in `Meta.constraints`, expressed as `Q(source_recording__isnull=False) | Q(source_span__isnull=False) | Q(source_media__isnull=False)`. Do **not** implement this in `save()` — `bulk_create()` bypasses `save()`, and J-03 writes provenance in bulk, which is exactly the path that would slip through.

- The raw-SQL case in AC-2 is not pedantry. It is the assertion that proves the rule lives in PostgreSQL and not in Python, and it is the reason this story exists separately from B-01.
- `source_span` is `{recording_id, t_start, t_end}` matching the `Question.evidence` shape, so the review UI in K-01 can seek a player from either.
- `confidence` should be a bounded decimal with a check of its own ($0.0 \leq \text{confidence} \leq 1.0$). Cheap to add here, annoying to add later.

Test cases

File	Case	Proves
<code>apps/onboarding/tests/test_provenance_constraint.py</code>	<code>test_create_without_source_raises</code>	ORM path blocked
<code>apps/onboarding/tests/test_provenance_constraint.py</code>	<code>test_bulk_create_without_source_raises</code>	The path J-03 actually uses is blocked
<code>apps/onboarding/tests/test_provenance_constraint.py</code>	<code>test_raw_sql_insert_without_source_raises</code>	The rule is in the database
<code>apps/onboarding/tests/test_provenance_constraint.py</code>	<code>test_confirmed_row_not_overwritten</code>	PG-06 at the model layer
<code>apps/onboarding/tests/test_migrations.py</code>	<code>test_constraint_replay</code>	Reverse and re-apply is clean

B-06 • Provenance APIs — list, confirm, edit

2 pts • Developer • Depends on B-05 • Blocks K-01, K-02, L-02 • Design §10.2, §12 EVT-109, §15 • Requirements FR-REV-02, FR-REV-03

As an Admin, I **want** provenance list, confirm and edit endpoints with role enforcement, **so that** every review action is recorded and can feed the learning loop.

Acceptance criteria

AC-1 • Provenance lists group by wizard page

- **Given** a session with provenance rows across several models and fields
- **When** `GET /api/v1/onboarding/sessions/{id}/provenance` is called
- **Then** rows are returned grouped by the wizard page each field belongs to, each carrying classification, confidence, extracted value, final value, status and its source reference

AC-2 • The KEY / SECONDARY asymmetry is enforced

- **Given** the §15 matrix, in which an Editor may run extraction but may not confirm a KEY field
- **When** an Editor confirms a `SECONDARY` field
- **Then** it succeeds and the row moves to `CONFIRMED`
- **When** the same Editor confirms a `KEY` field
- **Then** the response is 403 with `ERR-03` and the row is unchanged

AC-3 • Edits record the human value distinctly from the extracted one

- **Given** a row with `extracted_value = "Acme Ltd"`
- **When** an Admin edits it to `"Acme Limited"`

- Then `final_value` becomes "Acme Limited", `extracted_value` is preserved unchanged, `status` becomes `EDITED`, and `reviewed_by` and `reviewed_at` are set
- And `EVT-109` is emitted carrying the edit distance and the field name, and carrying neither value

AC-4 • Confirmation is idempotent

- Given a row already `CONFIRMED`
- When `confirm` is called again
- Then the response is 200 and nothing changes, rather than a second event being emitted

Technical notes

- Preserving `extracted_value` on edit is what makes L-02's golden-dataset candidates possible. If the edit overwrites it, the flywheel has no signal. This is the single most important line in the endpoint.
- `EVT-109` carrying edit *distance* but not the values is a §12 requirement enforced by `tests/test_events_no_pii.py`. Compute Levenshtein on the server and discard the strings.
- The wizard-page grouping needs a field-to-page map. Put it in `apps/onboarding/field_map.py` as data shared with J-02's extraction target list, so extraction and review cannot disagree about where a field belongs.

Test cases

File	Case	Proves
<code>apps/onboarding/tests/test_provenance_api.py</code>	<code>test_editor_cannot_confirm_key</code>	The asymmetry holds
<code>apps/onboarding/tests/test_provenance_api.py</code>	<code>test_edit_preserves_extracted_value</code>	The flywheel signal survives
<code>apps/onboarding/tests/test_provenance_api.py</code>	<code>test_confirm_idempotent</code>	No duplicate events
<code>tests/test_events_no_pii.py</code>	<code>test_evt_109_carries_no_values</code>	Only distance and field name leave the service

B-07 • Consent API

2 pts • Developer • Depends on B-02, B-04 • Blocks B-08, F-01 • Design §5.1 IG-08, §10.2 • Requirements FR-REC-01, NFR-PRIV-01

As an Admin, I want consent recorded through an API before recording can start, so that recording is lawful and the lawfulness is provable after the fact.

Acceptance criteria

AC-1 • Consent captures who, how and what

- Given a session in `READY`
- When `POST /api/v1/onboarding/sessions/{id}/consent` is called with subject name, method and scope
- Then a `ConsentRecord` is created with `granted_by` set from the authenticated user and `granted_at` set server-side
- And a client-supplied `granted_at` is ignored rather than trusted

AC-2 • The session exposes consent state

- **Given** a session with and without consent
- **When** the session is read
- **Then** the response carries a `consent` object with `granted`, `granted_at`, `method` and `scope`, or `granted: false`

AC-3 • Revocation is immediate and visible

- **Given** a session with granted consent
- **When** `DELETE /.../consent` is called
- **Then** `revoked_at` is set, the session's consent state flips to not granted, and any open live session for that company is closed with 4403
- **And** the tenant's retention workflow is notified so M-03's erasure path can act within the configured window

AC-4 • Consent is per session, not per tenant

- **Given** a tenant with a previously consented session
- **When** a new session is created for the same company
- **Then** consent is not inherited — the new session reports `granted: false`

Technical notes

- AC-4 is the one people get wrong. Consent attaches to the specific conversation being recorded, not to the customer relationship. Inheriting it would be both a GDPR problem and a product problem.
- The 4403 close on revocation requires the API to reach the agent. Publish an `agent.commands` message rather than calling the agent synchronously; a revocation must succeed even if the agent is down.
- `scope` is JSON so it can grow (audio, transcript, captured media, retention period) without a migration. Define the v1 shape in the serializer.

Test cases

File	Case	Proves
<code>apps/onboarding/tests/test_consent_api.py</code>	<code>test_granted_at_server_side</code>	Client cannot backdate consent
<code>apps/onboarding/tests/test_consent_api.py</code>	<code>test_consent_not_inherited</code>	Per-session semantics
<code>apps/onboarding/tests/test_consent_api.py</code>	<code>test_revocation_closes_live_session</code>	Revocation has teeth

B-08 • Recording lifecycle APIs

2 pts • Developer • Depends on B-07 • Blocks F-02, I-01 • Design §5.1 IG-08, §10.2 • Requirements FR-REC-02, FR-LIB-01

As an Admin, I want open and stop endpoints that create and finalise `MeetingRecording` rows, so that every record cycle is tracked and the library has something to list.

Acceptance criteria

AC-1 • Opening without consent is refused server-side

- **Given** a session with no `ConsentRecord`
- **When** `POST /.../recordings` is called

- **Then** the response is 403 with `ERR-09` and no row is created
- **And** this holds even when the client's UI would have prevented it — the gate is server-side, per IG-08

AC-2 • Stop finalises exactly the row it opened

- **Given** an open recording
- **When** `POST /.../recordings/{rid}/stop` is called
- **Then** `stopped_at`, `duration_s` and `status = UPLOADED` are set on that row only
- **And** stopping an already-stopped recording returns 200 without changing the duration

AC-3 • The list endpoint serves the library

- **Given** a session with several recordings and captured media
- **When** `GET /.../recordings` is called
- **Then** recordings are returned newest-first with duration, status, summary presence and the linked audio asset
- **And** a Viewer receives the same list read-only

AC-4 • An abandoned recording does not stay open forever

- **Given** a recording opened and never stopped
- **When** the stuck-session sweeper from Design §20 runs
- **Then** the row is closed with `status = FAILED` after the configured timeout, and the operator sees it in the library rather than the row vanishing

Technical notes

- `duration_s` should be computed from the actual audio asset once F-03's upload finalises, not from wall-clock between open and stop. A paused upload or a slow network makes wall-clock wrong, and the library shows the wrong number to the operator.
- `status = UPLOADED` here, not `TRANSCRIBED`. The transcript arrives asynchronously from F-05 or the F-06 backfill, and the library must be able to show "transcribing" honestly.
- The sweeper in AC-4 is implemented in M-05; this story only asserts the row shape it needs and adds the query index on `(status, started_at)`.

Test cases

File	Case	Proves
<code>apps/onboarding/tests/test_recording_api.py</code>	<code>test_open_without_consent_403</code>	IG-08 is enforced at the API, not only the UI
<code>apps/onboarding/tests/test_recording_api.py</code>	<code>test_stop_is_idempotent</code>	Double-stop does not corrupt duration
<code>apps/onboarding/tests/test_recording_api.py</code>	<code>test_list_newest_first_viewer_readonly</code>	The library contract holds for all roles

6. Epic C — PREP: Research and Questionnaire via Chat

The operator prepares in the chat interface they already use. Nothing in this epic introduces a new place to work; the Onboarding Interface only *displays* the result.

C-01 • Route onboarding-prep chat turns to the agent

3 pts • Developer • Depends on A-06 • Blocks C-02 • Design §2.1 PREP, §10.2.1 • Requirements FR-PREP-01, FR-PREP-02

As an Admin, I **want** my chat conversations about onboarding preparation routed to the Onboarding Intelligence Agent, **so that** preparation happens where I already work rather than in a new tool.

Acceptance criteria

AC-1 • Prep intents reach the agent

- **Given** the existing pipeline composer in `ai-brand-automator`
- **When** a chat turn is classified as an onboarding-prep intent
- **Then** it is dispatched to `POST /v1/execute` on port 8120 with an `X-Service-Token` header and the tenant context
- **And** non-prep turns continue to route exactly as they do today

AC-2 • Multi-turn context is maintained

- **Given** a prep conversation spanning several turns
- **When** the operator refers back to something said two turns earlier ("make the third question deeper")
- **Then** the agent resolves the reference correctly, because conversation state is keyed on the chat session id in Redis DB 27 per Design §14

AC-3 • Agent unavailability degrades honestly

- **Given** the agent returning 503 or timing out
- **When** the operator sends a prep turn
- **Then** the chat replies with a specific message naming preparation as temporarily unavailable and suggesting the manual path, rather than a generic error or a silent hang
- **And** the failure is recorded as `ERR - 13` with the circuit breaker for the backend dependency updated

Technical notes

- `X-Service-Token` verification lives in `app/api/deps.py`, mirroring the fleet. Do not invent a new auth scheme for this service.
- The `POST /v1/execute` request and response bodies are given in full in Design §10.2.1. Implement them exactly; C-02 through C-04 all ride this envelope.
- Conversation state key is `oia:v1:{tenant}:chat:{chat_session_id}` with the TTL from Design §14. Prep conversations are not eternal, and an untimed key on a shared Redis is a slow leak.

Test cases

File	Case	Proves
<code>tests/test_backend_contracts.py</code>	<code>test_execute_envelope</code>	Request and response match §10.2.1

File	Case	Proves
tests/ test_redis_key_isolation.py	test_chat_key_tenant_scoped	No unscoped chat key can be built
apps/chat/tests/test_routing.py	test_non_prep_intents_unchanged	Existing chat behaviour is untouched

C-02 • Research the business from operator hints (SKL-OIA-01)

3 pts • Developer • Depends on C-01 • Blocks C-03 • Design §8.1 SKL-OIA-01, §18.2 • Requirements FR-PREP-03

As an Admin, I **want** the agent to research the business from a name, a website and a few notes, **so that** the questions it drafts are grounded in something rather than generic.

Acceptance criteria

AC-1 • A structured brief is produced with sources

- **Given** a business name, an optional website and free-text notes
- **When** SKL-OIA-01 runs
- **Then** a `BusinessResearchBrief` is returned containing established facts, likely competitors, digital presence and explicit unknowns
- **And** every asserted fact carries a source URL; a fact the agent cannot source is placed under unknowns rather than stated

AC-2 • The brief is persisted against the session

- **Given** a completed brief
- **When** the operator returns to the conversation later, or opens the Onboarding Interface
- **Then** the same brief is available without re-running research

AC-3 • Search failure degrades to a flagged brief

- **Given** the Tavily circuit breaker in the open state
- **When** research is requested
- **Then** the agent produces a brief from the operator-provided information only, marked `degraded: true` with the reason
- **And** the chat response says so plainly, so the operator knows the questions are less grounded than usual

Technical notes

- The skill lives in `app/skills/research_business.py` and its declaration in `config/skills.yaml` sets `circuit_breaker_dependency: "tavily"` so the breaker wiring is declarative.
- Unknowns are not a consolation prize — they are the highest-value output of this skill, because SKL-OIA-02 turns them directly into questions. Prompt for them explicitly.
- Cache briefs by `(tenant, normalised business name)` for a short TTL. Operators re-run prep several times while tuning question count, and each re-run is otherwise a fresh round of paid search calls.

Test cases

File	Case	Proves
tests/test_guardrails.py	test_og_unsourced_fact_moves_to	OG grounding applies to research

File	Case	Proves
	<code>_unknowns</code>	too
<code>tests/test_circuit_breakers.py</code>	<code>test_tavily_open_produces_degraded_brief</code>	Degradation is flagged, not silent
<code>tests/e2e/test_prep_to_questionnaire.py</code>	Brief step	The brief is reachable from the session

C-03 • Generate a questionnaire to count and depth (SKL-OIA-02)

3 pts • Developer • Depends on C-02 • Blocks C-04 • Design §8.1 SKL-OIA-02 • Requirements FR-PREP-04, FR-PREP-08

As an Admin, I **want** a questionnaire generated to the number of questions I ask for and the depth I choose, **so** that preparation fits the meeting I actually have booked.

Acceptance criteria

AC-1 • Count and depth are honoured

- **Given** a request for 12 questions at `standard` depth
- **When** SKL-OIA-02 runs
- **Then** exactly 12 questions are returned
- **And** the same request at `deep` produces questions that probe mechanism and evidence rather than facts, verified against a rubric fixture

AC-2 • Every question is tagged for a workflow and a target field

- **Given** a generated questionnaire
- **When** it is inspected
- **Then** each question carries `workflow_target` in `{WF1, WF2, WF3}` and, where applicable, a `target_field` naming the `Company` field it feeds
- **And** questions covering WF3 creative reuse — existing ads, business photography, brand assets in use — are present, not only WF1 and WF2 questions

AC-3 • Coverage is visible before the meeting

- **Given** a generated set
- **When** it is returned to chat
- **Then** a coverage summary shows which workflow areas are well covered and which are thin, so the operator can ask for more before approving

AC-4 • It is stored as DRAFT

- **Given** a generated set
- **When** generation completes
- **Then** a `Questionnaire` row exists in `DRAFT` at `version = 1` with its `Question` children ordered

Technical notes

- AC-2's second clause is the one the requirement review added specifically: preparation is not scoped to the five-page wizard. The skill's prompt must carry the WF3 asset-collection intent, or the generated set silently reverts to brand-strategy questions only.

- `target_field` must be drawn from the shared `apps/onboarding/field_map.py` vocabulary introduced in B-06, not invented per generation, or J-02 cannot join questions to fields.
- The rubric fixture in AC-1 is a small checked-in file of depth exemplars. Without it, "deep" is untestable and will drift with every prompt edit.

Test cases

File	Case	Proves
<code>tests/test_skills_yaml.py</code>	SKL-OIA-02 declaration	Input schema declares count and depth as required
<code>tests/e2e/test_prep_to_questionnaire.py</code>	<code>test_count_honoured</code>	Asking for 12 yields 12
<code>apps/onboarding/tests/test_questionnaire.py</code>	<code>test_target_fields_in_vocabulary</code>	No invented field names
<code>apps/onboarding/tests/test_questionnaire.py</code>	<code>test_wf3_coverage_present</code>	Asset-collection questions are generated

C-04 • Refine and approve interactively (SKL-OIA-03)

3 pts • Developer • Depends on C-03 • Blocks C-05, F-04 • Design §8.1 SKL-OIA-03, §9.4 • Requirements FR-PREP-05, FR-PREP-06

As an Admin, I **want** to edit, regenerate and reorder questions and then approve the set, **so that** the final questionnaire is mine rather than the model's.

Acceptance criteria

AC-1 • Refinement works per question and per set

- **Given** a DRAFT questionnaire
- **When** the operator asks to rewrite question 4, drop question 7, or regenerate the whole set at greater depth
- **Then** each operation applies correctly and leaves ordering contiguous

AC-2 • Approval freezes a version

- **Given** a DRAFT at `version = n`
- **When** the operator approves
- **Then** `status = APPROVED`, `approved_by` and `approved_at` are set, and the question set becomes immutable
- **And** the session moves `PREPARING` → `READY` per Design §9.4

AC-3 • Re-approval creates a new version rather than mutating

- **Given** an APPROVED questionnaire
- **When** the operator edits it again
- **Then** a new DRAFT at `version = n + 1` is created and the approved version is preserved intact
- **And** approving the new draft supersedes the old one without deleting it

AC-4 • A meeting cannot start without approval

- **Given** a session whose questionnaire is DRAFT
- **When** a live session is attempted

- **Then** it is refused — the IG-10 gate — closing with 4403 and a message naming the missing approval

Technical notes

- AC-3's versioning matters more than it looks. An operator who reruns prep after a first meeting must not lose the record of what was asked in that meeting; the evidence spans on **Question** point at a specific version's rows.
- Reuse: a questionnaire marked **is_template** can be cloned for another company in the same tenant (Design D-05). Implement the clone here — it is a few lines while the code is open, and it is the feature operators ask for after their third onboarding.
- The **PREPARING** → **READY** transition must go through B-04's transition table rather than being set directly, or the state machine has a hole.

Test cases

File	Case	Proves
apps/onboarding/tests/test_questionnaire.py	test_approval_freezes_version	Approved rows are immutable
apps/onboarding/tests/test_questionnaire.py	test_reapproval_creates_new_version	History survives
tests/test_ws_handshake.py	test_draft_questionnaire_rejected_4403	IG-10 gate holds

C-05 • Show approved questions in the Onboarding Interface

2 pts • Frontend • **Depends on** C-04, E-01 • **Blocks** E-02 • **Design** §11 **QuestionChecklist** • **Requirements** FR-PREP-07

As an Admin, I **want** my approved questions visible in the Onboarding Interface, **so that** they are in front of me when the meeting starts.

Acceptance criteria

AC-1 • The checklist renders the approved set

- **Given** a session with an APPROVED questionnaire
- **When** the operator opens the Onboarding Interface
- **Then** questions render in order with their workflow tags and unchecked checkboxes

AC-2 • The empty state leads somewhere useful

- **Given** a session with no approved questionnaire
- **When** the interface opens
- **Then** the checklist shows an empty state linking directly into the chat prep flow, rather than an empty box

AC-3 • Re-approval is reflected

- **Given** an operator who re-approves a new version while the interface is open
- **When** the view refreshes
- **Then** the new version's questions are shown, and any local checkbox state for questions that no longer exist is discarded cleanly

Technical notes

- `QuestionChecklist` is specified in Design §11. Keep checkbox state server-authoritative from the start — G-03 will drive checkboxes from sufficiency signals, and a component built around local state has to be rewritten then.
- Workflow tags should be visually quiet. They matter to the operator's sense of coverage but they must not compete with the question text during a live conversation.

Test cases

File	Case	Proves
frontend/__tests__/QuestionChecklist.test.tsx	test_renders_approved_set	Order and tags correct
frontend/__tests__/QuestionChecklist.test.tsx	test_empty_state_links_to_prep	The dead end is not a dead end
tests/e2e/test_prep_to_questionnaire.py	Final step	Chat approval is visible in the interface

7. Epic D — Calendar and Scheduling

The in-app calendar is the source of truth for onboarding meetings; Google Calendar is an optional overlay. This ordering — build ours first, sync second — is deliberate, because it means the feature works for a tenant who never connects Google.

D-01 • In-app onboarding calendar

3 pts • Frontend + Developer • Depends on E-01 • Blocks D-02 • Design §11 [CalendarPane](#), §10.2 • Requirements FR-CAL-01

As an Admin, I want a calendar inside the Onboarding Interface that manages onboarding meetings, so that scheduling works with zero external setup.

Acceptance criteria

AC-1 • Meetings are created, edited and cancelled against a session

- **Given** the calendar pane in month or week view
- **When** the operator creates a meeting
- **Then** an [OnboardingSession](#) is created or linked, and the meeting shows the company, the operator and the scheduled window
- **And** editing or cancelling updates or releases the session accordingly

AC-2 • Timezones are correct, not approximately correct

- **Given** an operator in one timezone scheduling with a brand owner in another
- **When** the meeting is created and then viewed by a colleague in a third timezone
- **Then** every view shows the same instant rendered in the viewer's local zone
- **And** a meeting spanning a daylight-saving transition renders with the correct local time on both sides

AC-3 • Roles are enforced

- **Given** the §15 matrix
- **When** each role opens the calendar
- **Then** Owner, Admin and Editor may create and edit; Viewer sees a read-only calendar

Technical notes

- Store instants in UTC with an explicit IANA timezone alongside, never a fixed offset. AC-2's DST clause exists because offset-storing is the standard bug here and it surfaces twice a year, in production, on a customer call.
- Do not build a general-purpose calendar. This one shows onboarding meetings and nothing else; scope creep here is unbounded.

Test cases

File	Case	Proves
<code>apps/onboarding/tests/test_calendar.py</code>	<code>test_dst_transition_rendering</code>	Instants survive DST
<code>frontend/__tests__/CalendarPane.test.tsx</code>	<code>test_viewer_readonly</code>	RBAC at the component level

D-02 • Connect Google Calendar via OAuth

3 pts • Developer • Depends on D-01 • Blocks D-03 • Design §19 • Requirements FR-CAL-02, NFR-SEC-03

As an Admin, I **want** to connect my Google Calendar, **so that** my existing calendar stays the source of truth for my own availability.

Acceptance criteria

AC-1 • Only Admins and Owners can connect

- **Given** the §15 matrix
- **When** an Editor or Viewer attempts to initiate the OAuth flow
- **Then** it is refused with 403

AC-2 • Tokens are stored encrypted, Django-side, per tenant

- **Given** a completed OAuth exchange
- **When** tokens are persisted
- **Then** they are encrypted at rest at `secrets/<tenant_id>/google_calendar` and **never** transmitted to the agent service
- **And** an audit event records the connection with the acting user, carrying no token material

AC-3 • Disconnect actually revokes

- **Given** a connected calendar
- **When** the operator disconnects
- **Then** the refresh token is revoked at Google, the stored secret is deleted, and sync stops within one cycle

AC-4 • Failures are legible

- **Given** an expired or revoked grant
- **When** a sync attempt fails
- **Then** the calendar pane shows a specific reconnect prompt rather than silently going stale

Technical notes

- Design §19 is explicit that the agent never holds calendar OAuth credentials. Keeping the integration entirely Django-side means the agent's blast radius excludes the operator's personal calendar — worth the extra hop.
- Request the narrowest scope that supports read plus event creation. `calendar.events` rather than `calendar`.
- Token refresh should be a scheduled job, not lazy-on-use, so that a stale grant surfaces as an actionable notification rather than as a failed sync at the moment the operator needs it.

Test cases

File	Case	Proves
<code>apps/integrations/tests/test_google_calendar.py</code>	<code>test_editor_cannot_connect</code>	Admin-only
<code>apps/integrations/tests/test_google_calendar.py</code>	<code>test_disconnect_revokes_upstream</code>	Revocation is real
<code>apps/integrations/tests/test_secrets.py</code>	<code>test_no_token_in_logs_or_events</code>	Audit carries no material

D-03 • Two-way sync with Google Calendar

2 pts • Developer • Depends on D-02 • Blocks — • Design §11 CalendarPane • Requirements FR-CAL-03

As an Admin, I **want** externally scheduled meetings to appear here and in-app meetings to appear in Google, **so that** I do not maintain two calendars.

Acceptance criteria

AC-1 • External events appear within the lag budget

- **Given** a connected calendar
- **When** a meeting is created in Google
- **Then** it appears in the calendar pane within 5 minutes, or immediately on manual refresh

AC-2 • In-app meetings are pushed outward

- **Given** a meeting created in the app
- **When** sync runs
- **Then** a corresponding Google event exists with the same instant, title and attendees, and subsequent edits update rather than duplicate it

AC-3 • Sync failure is never fatal

- **Given** Google returning errors or rate limits
- **When** sync runs
- **Then** the in-app calendar continues to function fully, the failure is surfaced as a non-blocking notice, and the next cycle retries with backoff

AC-4 • Conflicts have a documented resolution

- **Given** the same meeting edited on both sides between syncs
- **When** sync reconciles
- **Then** the documented rule applies (in-app wins for onboarding-owned events; Google wins for externally created ones), and the losing change is recorded rather than discarded silently

Technical notes

- Use a sync token rather than polling a date range; full re-fetch on every cycle will hit quota with a handful of tenants.
- Tag app-created Google events with a private extended property so AC-4's ownership rule is decidable rather than heuristic.

Test cases

File	Case	Proves
apps/integrations/tests/test_google_calendar.py	test_sync_failure_non_fatal	The app survives Google being down
apps/integrations/tests/test_google_calendar.py	test_conflict_resolution_rule	The documented rule is the implemented rule

8. Epic E — Onboarding Interface Shell

Two stories, deliberately small, landing early. Their purpose is to give every later frontend story a place to attach rather than to deliver function.

E-01 • The onboarding entry point and landing view

3 pts • Frontend • **Depends on** B-04 • **Blocks** C-05, D-01, E-02 • **Design** §11 [OnboardingHome](#) • **Requirements** FR-UI-01, NFR-COMPAT

As an Admin, I **want** the onboarding icon to open a new Onboarding Interface, **so that** the meeting-driven flow is the front door without the old one being bricked up.

Acceptance criteria

AC-1 • The route renders the landing composition

- **Given** an authenticated Admin
- **When** [/onboarding](#) is opened
- **Then** the calendar pane, the sessions list and clear entry points to prepare, meet and go to the forms all render

AC-2 • The old path still works

- **Given** an existing bookmark or link to step 1 of the five-page wizard
- **When** it is opened
- **Then** the wizard loads and works exactly as before, with no meeting required at any point
- **And** this is asserted by an automated test, not by inspection

AC-3 • Viewer gets a real read-only variant

- **Given** a Viewer
- **When** they open [/onboarding](#)
- **Then** they see sessions and recordings but no create, edit or record affordances — the buttons are absent, not merely disabled with a hidden working endpoint behind them

Technical notes

- AC-2 is NFR-COMPAT's most visible expression and it is the thing most likely to be broken accidentally by a router change. Put its test in the e2e suite from this story onward.
- "Go to onboarding forms" must deep-link to the first page of the wizard for the session's company, carrying the session id so K-01's review links can return the operator to where they were.

Test cases

File	Case	Proves
tests/e2e/ test_process_to_review.py	test_manual_wizard_path_intact	NFR-COMPAT
frontend/__tests__/ OnboardingHome.test.tsx	test_viewer_affordances_absent	Read-only is real

E-02 • Meeting view layout

2 pts • Frontend • Depends on E-01, C-05 • Blocks F-01, G-03, H-01, I-01 • Design §11 MeetingView • Requirements FR-LIVE-02

As an Admin, I **want** the meeting view's split layout — questions above, agent feedback below, recordings and captures on the right — **so that** the live experience has its shape before anything is wired to it.

Acceptance criteria

AC-1 • The three regions exist and are independently scrollable

- **Given** the meeting view with placeholder data
- **When** it renders
- **Then** the question checklist occupies the upper pane, the agent feedback stream the lower pane, and the recordings and captures rail the right side, each scrolling without moving the others

AC-2 • The layout survives a real laptop

- **Given** a 13-inch screen at typical browser zoom
- **When** the view renders with twenty questions and a running feedback stream
- **Then** all three regions remain usable without horizontal scrolling, and the question text is not truncated to a single line

AC-3 • Nothing steals focus

- **Given** the operator typing a note or interacting with a checkbox
- **When** placeholder feedback arrives in the lower pane
- **Then** focus, scroll position and selection are unchanged
- **And** the feedback pane does not auto-scroll away from a position the operator has manually scrolled to

Technical notes

- AC-3 is called out in Design §11 as a hard rule: nothing the agent produces may steal focus. It is much cheaper to build this in now than to retrofit it once G-02 through G-06 are all pushing updates into the pane.
- Keep checkbox interactions in local state in this story only. G-03 replaces that with server-authoritative state, and the component boundary should already anticipate it.

Test cases

File	Case	Proves
frontend/__tests__/MeetingView.test.tsx	test_focus_preserved_on_stream_update	The focus rule holds
frontend/__tests__/MeetingView.test.tsx	test_no_autoscroll_after_manual_scroll	The operator stays in control

9. Epic F — Recording and Real-Time Transcription

This epic builds the audio path end to end: consent, capture, durable spool, socket, speech recognition, and the failure mode. It is sequenced so that every story leaves the system in a state where **the meeting still works**. F-02 gives you a recording with no transcript. F-05 adds the transcript. F-06 makes sure that losing the transcript does not lose the recording. At no point between these stories is there a build where an operator could start a meeting and end up with nothing.

The order matters for a second reason. Design §4.3 makes durability a property of the spool, not of the socket — audio reaches GCS whether or not STT, the LLM or the network behaves. Building the spool (F-03) before the socket (F-04) means the socket is never the only thing standing between a meeting and its recording.

F-01 • Consent capture before the microphone opens

2 pts • Full-stack • **Depends on** B-07, E-02 • **Blocks** F-02 • **Design** §5.1 IG-08, §10.1 [ConsentRecord](#) • **Requirements** FR-GDPR-01, FR-REC-01

As an Admin running an onboarding meeting, **I want** to record the brand owner's consent before the microphone can be enabled, **so that** we never hold a recording we had no lawful basis to make.

Acceptance criteria

AC-1 • The record button is inert until consent exists

- **Given** a session in [READY](#) with no [ConsentRecord](#)
- **When** the operator opens the meeting view
- **Then** the record control is visible but disabled, with the reason stated inline — "Record consent to enable recording" — rather than as an unexplained grey button
- **And** clicking it opens the consent modal rather than doing nothing

AC-2 • Consent is captured as a record, not a checkbox

- **Given** the consent modal
- **When** the operator enters the subject's name, selects the method ([VERBAL](#) or [WRITTEN](#)), and confirms
- **Then** a [ConsentRecord](#) is created with [granted_at](#) set server-side, the operator's user id as witness, and the scope covering recording, transcription and processing
- **And** [EVT-101 onboarding.consent.verified](#) is emitted carrying [consent_id](#), [method](#) and [subject_name_hash](#) — never the subject's name

AC-3 • The gate is enforced at the agent, not only in the browser

- **Given** a session with no consent
- **When** a WebSocket connection is attempted directly against [/v1/live/{session_id}](#), bypassing the UI
- **Then** the agent closes the socket with [4403](#) and emits [EVT-004](#) with [rule_id: IG-08](#)

AC-4 • Revocation is visible immediately

- **Given** an active meeting whose consent is revoked from the session detail page
- **When** the revocation lands
- **Then** the meeting view shows a blocking banner, the record control returns to disabled, and any open socket is closed with [4403](#) within 5 seconds

Technical notes

- The gate lives in `app/logic/guardrails.py` as IG-08 and is evaluated on WS accept, on `POST /v1/process`, and on every `start` control frame — three places, one function. A single evaluation point on connect is not sufficient because B-07's revocation path can fire mid-session.
- The modal collects a name for the record; only its hash reaches the event stream. Store the plaintext name on `ConsentRecord` in the tenant-scoped database, which is the subject-access-request surface, and nowhere else.
- Consent is per session and is never inherited from a previous session with the same company — B-07 established that rule and this story is where it becomes visible to a user. If the operator has run three meetings with this brand owner, they record consent three times.

Test cases

File	Case	Proves
<code>tests/test_guardrails.py</code>	<code>test_ig08_blocks_ws_without_consent</code>	The gate is server-side
<code>tests/test_ws_handshake.py</code>	<code>test_close_4403_on_missing_consent</code>	The right close code
<code>tests/test_events_no_pii.py</code>	<code>test_evt101_carries_hash_not_name</code>	The event stream stays clean
<code>tests/e2e/test_live_meeting.py</code>	<code>test_revocation_closes_open_socket</code>	Revocation is not advisory

Risk. The revocation path is the part teams skip. Write AC-4's test before AC-2's implementation if you want it to exist.

F-02 • Microphone capture in the browser

3 pts • Frontend • **Depends on** F-01, E-02 • **Blocks** F-03 • **Design** §4.3, §11 `RecorderControl` • **Requirements** FR-REC-02, NFR-PERF-01

As an Admin, I **want** to start and stop microphone recording from the meeting view, **so that** the meeting is captured without me leaving the tool I am running it from.

Acceptance criteria

AC-1 • Permission is requested once and handled honestly

- **Given** an operator who has not granted microphone permission to the origin
- **When** they press record
- **Then** the browser permission prompt is triggered, and a denial produces an explicit in-app state — "Microphone blocked. Enable it in your browser's site settings and press record again" — with the record control returned to its ready state, not a spinner

AC-2 • Capture produces the format the pipeline expects

- **Given** permission granted
- **When** recording starts
- **Then** `MediaRecorder` is configured for opus at 48 kHz with a 20 ms frame interval, matching the STT adapter's expectation in Design §4.3
- **And** if the browser does not support the opus mime type, recording is refused with a named, testable error rather than silently falling back to a codec the adapter cannot read

AC-3 • Recording state is unambiguous at a glance

- **Given** an active recording
- **When** the operator glances at the view
- **Then** they see an unmistakable recording indicator with a running elapsed timer, and the browser tab title reflects the recording state so a backgrounded tab is still obvious

AC-4 • Stopping is safe

- **Given** an active recording
- **When** the operator presses stop, or closes the tab, or the page crashes
- **Then** the recorder flushes its final buffer and the local queue is preserved in memory for F-03 to drain
- **And** a tab close during recording triggers a `beforeunload` confirmation naming what is at risk

Technical notes

- Do not use the `MediaRecorder` default timeslice. Pass an explicit `timeslice` so `ondataavailable` fires on a predictable cadence; F-03's chunking depends on it.
- Elapsed time must come from the audio timeline, not `Date.now()`. A machine that sleeps mid-meeting will otherwise report a duration that does not match the audio, and B-08 already established that `duration_s` comes from the asset.
- Keep the recorder in a hook with no knowledge of the socket. F-03 and F-04 both consume its output queue, and the seam is what makes F-06's record-only mode possible without a rewrite.

Test cases

File	Case	Proves
<code>frontend/__tests__/RecorderControl.test.tsx</code>	<code>test_permission_denied_returns_to_ready</code>	No spinner traps
<code>frontend/__tests__/RecorderControl.test.tsx</code>	<code>test_unsupported_codec_refuses_loudly</code>	No silent format drift
<code>frontend/__tests__/RecorderControl.test.tsx</code>	<code>test_elapsed_from_audio_timeline</code>	Duration matches the asset

F-03 • Chunked resumable upload and durable spool

3 pts • Full-stack • **Depends on** F-02, B-08 • **Blocks** F-04, I-01 • **Design** §4.3 spool, §18.2 `gcs` breaker • **Requirements** FR-REC-03, NFR-REL-01

As a System, I want captured audio streamed to a resumable GCS upload independently of the analysis path, so that a recording survives every failure the assist features can suffer.

Acceptance criteria**AC-1 • Audio reaches durable storage in chunks, not at the end**

- **Given** an active recording
- **When** thirty seconds of audio have accumulated
- **Then** that chunk is appended to a resumable GCS upload session under `_landing/{tenant_id}/{uuid}_{recording_id}.opus`
- **And** the upload session id is persisted so a process restart can resume rather than restart

AC-2 • A network interruption does not lose audio

- **Given** a recording in progress
- **When** the network drops for 90 seconds and returns
- **Then** the queued chunks are uploaded in order on reconnection with no gap and no duplication in the resulting file
- **And** the operator sees a transient "Saving delayed" indicator, not an error

AC-3 • The GCS breaker degrades to local spool without stopping the meeting

- **Given** the `gcs` circuit breaker open
- **When** recording continues
- **Then** chunks spool to bounded local disk per Design §18.2 `LOCAL_DISK_SPOOL`, the user message "Upload delayed — recording continues locally" is surfaced from the breaker config rather than hardcoded, and the spool drains on breaker close
- **And** when the local bound is reached, recording stops gracefully with an explicit message rather than silently discarding audio

AC-4 • The recording row reflects reality

- **Given** a completed upload
- **When** the recording is finalised
- **Then** `MeetingRecording.status` moves to `UPLOADED`, `duration_s` is read from the audio asset, and a `BrandAsset` row is registered pointing at the GCS object
- **And** replaying the same finalisation with the same `Idempotency-Key` produces no second asset

Technical notes

- The spool is deliberately not the socket. Design §4.3 draws these as two arrows out of `ws.py` for a reason: F-06's degraded mode is only cheap because the durability path never depended on STT being up.
- Use GCS resumable uploads rather than composing many small objects. Compose semantics make AC-2's "no duplication" much harder to guarantee under retry.
- The bound in AC-3 should be expressed in minutes of audio, not megabytes, because that is the unit the operator-facing message needs.

Test cases

File	Case	Proves
<code>tests/test_circuit_breakers.py</code>	<code>test_gcs_open_spools_locally</code>	Degradation per §18.2
<code>tests/test_circuit_breakers.py</code>	<code>test_local_spool_bound_stops_gracefully</code>	No silent audio loss
<code>tests/test_idempotency.py</code>	<code>test_finalise_recording_is_idempotent</code>	Replay safety
<code>tests/e2e/test_live_meeting.py</code>	<code>test_network_drop_resumes_without_gap</code>	The core durability claim

Risk. AC-2 is the story's whole value and it cannot be proven by unit test alone. Budget for a fault-injection harness that can actually sever the connection.

F-04 • WebSocket session lifecycle

3 pts • Backend • Depends on A-02, A-05, F-01 • Blocks F-05, G-02, G-03 • Design §10.2.3, §4.3
[LiveSessionManager](#) • Requirements FR-LIVE-01, NFR-SEC-02

As a System, I want `WS /v1/live/{session_id}` to authenticate, authorise, admit exactly one socket per session, and survive reconnection, so that everything downstream can assume a well-formed stream.

Acceptance criteria

AC-1 • The handshake authenticates and authorises before accepting

- **Given** a connection attempt
- **When** the handshake is evaluated
- **Then** the JWT is validated, tenant is resolved, the role is checked (Owner, Admin or Editor), consent is verified via IG-08, and the session state is confirmed live-eligible — all before `accept()`
- **And** each failure maps to its own close code: `4401` invalid or expired JWT, `4403` consent missing or revoked, `4404` session not found or not live-eligible, `4409` another socket already live, `4429` rate limited

AC-2 • One live socket per session, decisively

- **Given** an open live socket for a session
- **When** a second socket for the same session connects
- **Then** the second is closed with `4409` and the first is untouched
- **And** the lock is held in Redis DB 27 with a TTL so a crashed process cannot lock a session out permanently

AC-3 • Reconnection resumes rather than restarts

- **Given** a socket that dropped at `seq 812`
- **When** the client reconnects and sends `{"type": "resume", "last_seq": 812}` within the resume window
- **Then** the server replays frames after 812 from the session buffer and continues the same `seq` series
- **And** a resume attempt beyond the window is answered with an explicit resync frame, not a silent gap in `seq`

AC-4 • Every downstream frame carries a monotonic `seq`

- **Given** any server-to-client frame
- **When** it is emitted
- **Then** it carries a `seq` that is strictly increasing for the life of the session across all frame types
- **And** the frame shapes match Design §10.2.3 exactly, validated against a shared schema module rather than hand-built dicts

Technical notes

- `app/api/ws.py` owns the socket; `app/logic/live_session.py` owns the state machine. Keeping the protocol out of the logic is what lets `test_ws_handshake.py` test close codes without spinning up STT.
- Frame models belong in `app/api/schemas.py` and should be Pydantic models shared with the tests, so a frame-shape change breaks a test rather than a browser.
- The session buffer backing AC-3 is a capped Redis list in DB 27 under `oia:v1:{tenant}:live:{session_id}:frames`. Cap it by count, and make the resume window a config value — A-02's spike output tells you what the gateway's real idle behaviour is, and that is the number this should be tuned against.

Test cases

File	Case	Proves
tests/test_ws_handshake.py	test_close_codes_map_one_to_one	\$10.2.3 conformance
tests/test_ws_handshake.py	test_second_socket_rejected_4409	Single-writer guarantee
tests/test_session_state.py	test_resume_replays_after_last_seq	Reconnection is real
tests/test_redis_key_isolation.py	test_live_keys_are_tenant_scope_d	Multi-tenancy holds
tests/property/test_segment_ordering.py	test_seq_strictly_increasing	The ordering invariant

F-05 • Streaming speech-to-text with diarization

5 pts • Backend • Depends on A-01, F-04 • Blocks G-01, F-06 • Design §4.3, §9.2, §8.1 SKL-OIA-04 • Requirements FR-LIVE-03, NFR-PERF-01

As a System, I want live audio relayed to Google Speech-to-Text v2 with two-speaker diarization and results surfaced as partial and final segments, so that the operator sees the conversation as it happens and the analysis loop has clean units to work on.

Acceptance criteria**AC-1 • Partial segments reach the UI within the budget and never touch the LLM**

- **Given** audio flowing through the socket
- **When** STT emits an interim result
- **Then** a `transcript.partial` frame is forwarded to the client with no model call and no redaction pass, arriving within 2 seconds of the corresponding speech per NFR-PERF-01
- **And** the measurement is instrumented browser-side and reported, not asserted by inspection

AC-2 • Final segments are diarized and ordered

- **Given** a two-speaker conversation
- **When** STT emits `is_final` results
- **Then** each becomes a `transcript.final` frame carrying `text`, `speaker`, `t_start`, `t_end` and `redaction_applied`
- **And** finals are emitted in non-decreasing `t_start` order even when STT delivers a correction out of order

AC-3 • The stream is restarted before the provider's limit, invisibly

- **Given** a meeting that runs past the streaming recognition duration limit
- **When** the limit approaches
- **Then** the adapter opens a new recognition stream and closes the old one with overlap, so no audio is dropped and no duplicate segment is emitted
- **And** the operator sees nothing — no gap, no flicker, no duplicated sentence

AC-4 • Segments are persisted redacted, displayed unredacted

- **Given** a finalised segment containing a phone number

- **When** it is processed
- **Then** the operator sees the segment as spoken, while what enters the Redis transcript buffer and every later prompt has been through IG-04
- **And** nothing unredacted is written to any store or sent to any model

Technical notes

- `app/providers/stt.py` is an adapter with one interface and two implementations from day one: the Google client and a fixture-driven fake that replays `tests/fixtures/two_speaker_2min.wav` timings. Every test above the adapter uses the fake. This is the difference between a five-minute test suite and a fifty-minute one.
- AC-3 is the story's hidden cost and the reason this is 5 points rather than 3. A-01's spike measured the limit; this story implements the rollover. Overlap-and-dedup at the boundary, keyed on `t_start` plus a normalised text shingle.
- AC-4 restates Design §4.3's second load-bearing property. The redaction call is SKL-OIA-16, invoked between `is_final` and the Redis write — not between Redis and the prompt, which would be too late.
- Diarization returns speaker tags, not identities. Map tag to role — operator versus subject — using who started the recording, and keep that mapping out of the event stream.

Test cases

File	Case	Proves
<code>tests/test_stt_adapter.py</code>	<code>test_partial_bypasses_llm_and_redaction</code>	The latency design
<code>tests/test_stt_adapter.py</code>	<code>test_stream_rollover_no_gap_no_dup</code>	Long meetings work
<code>tests/property/test_segment_ordering.py</code>	<code>test_finals_non_decreasing_t_start</code>	Ordering under correction
<code>tests/e2e/test_live_meeting.py</code>	<code>test_persisted_transcript_is_redacted</code>	AC-4 end to end

Risk. AC-1's 2-second budget is a claim about the network path as much as the provider. Measure it through Kong on Railway, not against a local socket, or the number is fiction.

F-06 • Degraded mode — recording continues when speech fails

2 pts • Full-stack • **Depends on** F-05, F-03 • **Blocks** G-03 • **Design** §18.2 `stt` breaker • **Requirements** FR-LIVE-06, NFR-REL-02

As an Admin, I **want** the meeting to keep recording and stay manually usable when speech recognition fails, **so that** a provider outage costs me assistance, not the meeting.

Acceptance criteria

AC-1 • The breaker opens and the mode changes without dropping audio

- **Given** five STT failures inside the 30-second window per Design §18.2
- **When** the breaker opens
- **Then** the session enters `RECORD_ONLY`, audio continues spooling through F-03 uninterrupted, and `EVT-011 agent.circuit.opened` is emitted with `dependency: stt`

AC-2 • The operator is told exactly what they lost and what they did not

- **Given** `RECORD_ONLY`
- **When** the banner renders
- **Then** it shows the configured message — "Live assist paused — recording continues. Transcript will be ready after the meeting." — read from `config/circuit_breakers.yaml`, not from a string in the component
- **And** an `error` frame with `code: ERR-07` and `recoverable: true` was what triggered it

AC-3 • Manual operation is fully available

- **Given** `RECORD_ONLY`
- **When** the operator works through the questionnaire
- **Then** every question can be checked off by hand, notes can be taken, and captures still work
- **And** manual checkmarks are marked as manually sourced so J-02 does not later treat them as evidence-backed

AC-4 • Recovery is automatic and announced

- **Given** a healthy STT provider after the reset timeout
- **When** the half-open probe succeeds twice per the success threshold
- **Then** live assist resumes, the banner clears, and the operator is told that transcription has resumed
- **And** the audio recorded during the degraded window is still transcribed in `PROCESS`, so the transcript has no hole

Technical notes

- AC-4's last clause is the one that makes the degraded mode acceptable rather than merely survivable: the spool has the audio, so `PROCESS` transcribes it in batch. Without that, `RECORD_ONLY` silently produces a partial transcript and nobody notices until review.
- Manual checkmarks write a `Question` state with no `evidence` entries. B-05's `CheckConstraint` means they cannot masquerade as extracted facts, which is exactly the protection you want here.

Test cases

File	Case	Proves
<code>tests/test_circuit_breakers.py</code>	<code>test_stt_open_enters_record_only</code>	The mode transition
<code>tests/e2e/test_degraded_stt.py</code>	<code>test_meeting_completes_without_stt</code>	The whole premise
<code>tests/e2e/test_degraded_stt.py</code>	<code>test_degraded_window_transcribed_in_process</code>	No hole in the transcript
<code>tests/test_guardrails.py</code>	<code>test_manual_check_has_no_evidence</code>	Manual is not evidence

10. Epic G — Live Meeting Assist

This is the epic the product is named for. Everything before it makes a recording; this makes the recording useful while the meeting is still happening.

Two constraints shape every story here. The first is the 5-second p95 feedback budget from Design §18.3, together with its unusual companion rule: **a late signal is dropped, not delivered**. Feedback about an answer given forty seconds ago is worse than no feedback, because the operator reads it as commentary on what was just said. The second is E-02's focus rule — nothing the agent produces may move the operator's cursor, scroll position or selection. Both are testable, and both are in the acceptance criteria below rather than in a wiki page nobody reads.

G-01 • PII redaction on finalised segments

3 pts • Backend • **Depends on** F-05 • **Blocks** G-02, M-02 • **Design** §5.1 IG-04, §8.3 SKL-OIA-16 • **Requirements** FR-GDPR-03, NFR-SEC-03

As a System, I want every finalised transcript segment redacted before it is buffered, stored or sent to a model, **so that** personal data spoken in a meeting never leaves the boundary it was spoken in.

Acceptance criteria

AC-1 • Redaction runs at the right point in the path

- **Given** a finalised segment containing a person's name, a phone number and an email address
- **When** it is processed
- **Then** SKL-OIA-16 runs before the Redis write, before any prompt assembly, and before any Kafka emission
- **And** the un-redacted text exists only in the frame already sent to the operator's browser and in no server-side store

AC-2 • The entity set is configured, not hardcoded

- **Given** the Presidio analyser configuration
- **When** redaction runs
- **Then** the recognised entity set covers at minimum person, phone, email, credit card, IBAN, national identifiers and physical address, and the set is declared in configuration so a tenant in a new jurisdiction is a config change
- **And** each redaction replaces the value with a typed placeholder — `[PERSON]`, `[PHONE_NUMBER]` — preserving sentence structure so downstream extraction still parses

AC-3 • Business identity is not collateral damage

- **Given** a segment reading "I'm Sarah and I founded Kelso Coffee in 2016"
- **When** redaction runs
- **Then** "Sarah" is redacted and "Kelso Coffee" is not
- **And** this case is a named fixture, because an over-eager recogniser that redacts the brand name destroys the entire point of the meeting

AC-4 • The event stream sees types, never values

- **Given** a redacted segment
- **When** EVT-103 is emitted
- **Then** it carries `recording_id`, `seq`, `redaction_applied` and `entity_types`: `["PERSON"]`

- **And** it carries neither the segment text nor any entity value, asserted by a test that scans the serialised payload for the fixture's known secrets

Technical notes

- `app/skills/redact_pii.py` wraps Presidio with the analyser loaded once at startup, not per call. The per-segment budget here is tens of milliseconds; a cold analyser load inside the hot path will blow F-05's latency budget outright.
- Keep an allowlist seeded from the session's `Company.name`, trading name and any known product names, and pass it as a deny-recognition hint. That is the mechanism behind AC-3.
- `redaction_applied` on the frame is what lets the operator understand why a later summary reads differently from what they heard. Do not drop it as noise.

Test cases

File	Case	Proves
<code>tests/test_guardrails.py</code>	<code>test_ig04_runs_before_buffer_write</code>	Ordering in the path
<code>tests/test_guardrails.py</code>	<code>test_company_name_survives_redaction</code>	AC-3, the expensive bug
<code>tests/test_events_no_pii.py</code>	<code>test_evt103_payload_has_no_values</code>	The event-stream boundary
<code>tests/e2e/test_gdpr_erasure.py</code>	<code>test_no_unredacted_text_in_any_store</code>	The claim, end to end

Risk. AC-3 is where this story goes wrong in production rather than in test. Seed the fixture set from real brand names with person-like tokens — "Kelso", "Marlow", "Sadie's" — not from synthetic examples.

G-02 • Streaming analysis mapped to prepared questions

3 pts • Backend • Depends on G-01, C-04 • Blocks G-03, G-05, G-06 • Design §4.3 batcher, §8.1 SKL-OIA-04, §9.2 • Requirements FR-LIVE-04

As a System, I want batched transcript segments analysed and mapped onto the approved questionnaire, so that every later signal — green marks, follow-ups, coverage — has a shared understanding of what has been answered.

Acceptance criteria

AC-1 • Batching is speaker-turn aware, not timer-driven

- **Given** a stream of finalised segments
- **When** the batcher accumulates
- **Then** analysis fires on a speaker change or a 3-second window, whichever comes first, per Design §4.3
- **And** a 400-millisecond mid-sentence fragment is never dispatched to the model on its own

AC-2 • Segments map to questions with evidence spans

- **Given** a batch covering an answer to question `q_07`
- **When** SKL-OIA-04 runs
- **Then** the result associates the batch with `q_07` and carries evidence as `[{recording_id, t_start, t_end}]` matching the `Question.evidence` shape from Design §10.1
- **And** a batch that maps to no prepared question is retained for G-05 rather than discarded

AC-3 • Analysis is bounded and drops late rather than delivering late

- **Given** an analysis call that exceeds its 5-second budget
- **When** the timeout fires
- **Then** the result is discarded, EVT-009 is emitted with the timeout error code, and nothing is sent to the client
- **And** the next batch is analysed normally — one slow call does not cascade into a backlog

AC-4 • The LLM breaker degrades to manual

- **Given** the `llm` breaker open
- **When** batches arrive
- **Then** analysis is skipped, the configured message "Suggestions paused. Check questions off manually — nothing is lost." is surfaced, and transcription continues unaffected

Technical notes

- The batcher belongs in `app/logic/live_session.py`, not in the skill. The skill should receive a well-formed window and have no opinion about how it was assembled.
- Design §9.2 notes this batching roughly quarters LLM call volume against a naive per-segment loop. That is a cost argument and a quality argument at once — record the observed call rate in the story's demo so the claim is checked rather than assumed.
- Pass the redacted window, the question list with `target_field`, and the running coverage state. Do not pass the full transcript; the window plus L2 working memory is the design's contract.

Test cases

File	Case	Proves
<code>tests/test_session_state.py</code>	<code>test_batch_fires_on_speaker_change</code>	AC-1
<code>tests/test_session_state.py</code>	<code>test_fragment_not_dispatched_alone</code>	No junk windows
<code>tests/test_circuit_breakers.py</code>	<code>test_llm_open_keeps_transcription</code>	Graceful degradation
<code>tests/e2e/test_live_meeting.py</code>	<code>test_answer_maps_to_question_with_span</code>	The core mapping

G-03 • Green signals from answer sufficiency

3 pts • Full-stack • **Depends on** G-02, F-04, E-02 • **Blocks** G-04, G-06, J-02 • **Design** §8.1 SKL-OIA-05, §10.2.3 • **Requirements** FR-LIVE-05

As an Admin, I **want** a question to turn green when the brand owner has actually answered it, **so that** I can see what is still missing without breaking eye contact to read a transcript.

Acceptance criteria**AC-1 • Sufficiency is scored, and the threshold is explicit**

- **Given** an analysed batch mapped to `q_07`
- **When** SKL-OIA-05 scores it
- **Then** a score at or above 0.7 produces a `green_signal` frame carrying `question_id`, `score` and the evidence spans, per Design §10.2.3

- **And** a score below 0.7 produces no green signal and routes to G-04 instead

AC-2 • The checkbox state is server-authoritative

- **Given** a green signal
- **When** the client renders it
- **Then** the checkbox state came from the server, and a page refresh mid-meeting restores exactly the same state
- **And** C-05 established this contract; this story is where the server actually becomes the source of truth

AC-3 • The operator can always override

- **Given** any question, green or not
- **When** the operator checks or unchecks it manually
- **Then** the override is persisted, sent as a `mark_question` control frame, and is never subsequently reverted by an agent signal
- **And** the question records that its state was manually set, distinguishing it from an evidence-backed green

AC-4 • Signals arrive without stealing focus

- **Given** the operator typing a note
- **When** a green signal arrives
- **Then** the checkbox updates, and focus, scroll position and selection are unchanged per E-02's AC-3

Technical notes

- The 0.7 threshold belongs in `config/skills.yaml` alongside the skill definition, not in the prompt and not in the component. L-04's tenant customization will want to move it per tenant, and a threshold buried in a prompt string cannot be moved.
- AC-3's "never subsequently reverted" is a real ordering hazard: a manual uncheck followed 200 ms later by an in-flight green signal must not resurrect the check. Version the question state and reject stale signals by version, rather than by timestamp.
- Evidence spans are what make K-01's review page auditable. A green with no span is a bug even if the score is 0.99.

Test cases

File	Case	Proves
<code>tests/test_session_state.py</code>	<code>test_manual_override_survives_stale_signal</code>	The ordering hazard
<code>tests/test_session_state.py</code>	<code>test_refresh_restores_server_state</code>	AC-2
<code>frontend/__tests__/MeetingView.test.tsx</code>	<code>test_green_signal_preserves_focus</code>	The focus rule
<code>tests/e2e/test_live_meeting.py</code>	<code>test_green_carries_evidence_span</code>	Auditability

G-04 • Targeted follow-up suggestions

2 pts • Full-stack • Depends on G-03 • Blocks L-03 • Design §8.1 SKL-OIA-06, §12.2 EVT-105 • Requirements FR-LIVE-07

As an Admin, I **want** a short list of follow-up questions when an answer was thin, **so that** I can dig deeper in the moment instead of discovering the gap a week later.

Acceptance criteria

AC-1 • Follow-ups appear only where they are earned

- **Given** a question scored below 0.7 by G-03
- **When** SKL-OIA-06 runs
- **Then** at most three follow-ups are produced for that question and delivered as a `followups` frame
- **And** a question with no answer attempt at all produces no follow-ups — the operator has not asked it yet, and prompting them to dig into an unasked question is noise

AC-2 • Follow-ups are specific to what was said

- **Given** the answer "we've been going a while, mostly local customers"
- **When** follow-ups are generated
- **Then** they reference the actual gap — the founding year, the customer definition — rather than restating the original question in different words
- **And** a fixture set of thin answers is scored against this by review, with the rubric recorded in the test as an explicit list of unacceptable outputs

AC-3 • Acceptance is measured

- **Given** a delivered follow-up
- **When** the operator marks it as asked, or the analysis loop detects it was asked
- **Then** EVT-105's `accepted` field is backfilled for that suggestion
- **And** the acceptance rate is queryable per prompt version, because that is the input \$17.3's flywheel needs

AC-4 • The pane stays calm

- **Given** follow-ups arriving during a fast exchange
- **When** they render in the lower pane
- **Then** superseded suggestions for a question that has since gone green are removed rather than accumulating
- **And** the pane never grows an unbounded list the operator has to scroll past

Technical notes

- AC-3 is the story's quiet value. Without acceptance backfill, L-03 has nothing to optimise SKL-OIA-06 against, and the follow-up prompt becomes the one skill whose quality can never be measured.
- Detecting "was asked" can reuse G-02's mapping: a subsequent operator turn whose text is close to a suggestion counts. Keep the similarity threshold conservative and let the manual mark be the reliable path.

Test cases

File	Case	Proves
<code>tests/test_session_state.py</code>	<code>test_no_followups_for_unasked_question</code>	AC-1's second clause
<code>tests/test_session_state.py</code>	<code>test_superseded_followups_removed</code>	The pane stays calm
<code>tests/test_kafka_roundtrip.py</code>	<code>test_evt105_accepted_backfill</code>	Measurability

G-05 • Ad-hoc question detection

3 pts • Backend • Depends on G-02 • Blocks J-02, K-01 • Design §10.1 [Question.origin](#), §8.1 SKL-OIA-04 • Requirements FR-LIVE-08

As an Admin, I **want** questions I ask off-script captured alongside the prepared ones, **so that** the best parts of a meeting — the parts I improvised — are not the parts we lose.

Acceptance criteria

AC-1 • Off-list questions are detected from the operator's speech

- **Given** the operator asking a question that maps to no prepared question
- **When** the batch is analysed
- **Then** a [Question](#) row is created with [origin](#): [ADHOC](#), the question text as spoken, and the answer's evidence spans attached as they arrive
- **And** it appears in the checklist visually distinguished from prepared questions

AC-2 • Detection uses the speaker tag, not guesswork

- **Given** a diarized stream
- **When** a question-shaped utterance appears
- **Then** it is only considered an ad-hoc question if it came from the operator's speaker tag
- **And** a question asked by the brand owner — "do you need our VAT number?" — is captured as a notable fact, not as an onboarding question

AC-3 • Ad-hoc questions get a workflow target

- **Given** a detected ad-hoc question and its answer
- **When** it is recorded
- **Then** [workflow_target](#) is set to WF1, WF2 or WF3, and [target_field](#) is set where the answer maps to a known field via [apps/onboarding/field_map.py](#)
- **And** where no field maps, the answer is still retained as evidence for J-02 rather than dropped

AC-4 • Notable facts are surfaced live

- **Given** the brand owner volunteering something significant and unprompted
- **When** it is detected
- **Then** a [notable_fact](#) frame is sent carrying the fact and its [workflow_target](#), per Design §10.2.3
- **And** the example the design gives — a second retail location opening in October, tagged WF3 — is a fixture in the test suite

Technical notes

- This story is why C-03 insisted [target_field](#) come from a shared [field_map.py](#). Prepared and ad-hoc questions must resolve fields through the same map or J-02 will treat them differently.
- The [ADHOC](#) origin value already exists on the model from B-01. This story populates it; do not add a parallel flag.
- Be conservative. A false ad-hoc question clutters the checklist during a live meeting, which is the most expensive place to add noise. Prefer missing one over inventing one, and say so in the prompt.

Test cases

File	Case	Proves
tests/test_session_state.py	test_subject_question_not_captured_as_adhoc	AC-2
tests/test_session_state.py	test_adhoc_resolves_target_field_via_map	Shared field map
tests/e2e/test_live_meeting.py	test_notable_fact_frame_wf3	The design's own example

G-06 • Live WF1/WF2/WF3 coverage checklist

3 pts • Full-stack • **Depends on** G-02, G-03 • **Blocks** J-01 • **Design** §8.1 SKL-OIA-09, §1.1 G-2 • **Requirements** FR-PREP-08, FR-LIVE-09

As an Admin, I **want** to see how well the meeting has covered each of the three workflows, **so that** I can spend the last ten minutes on the workflow that is thinnest instead of guessing.

Acceptance criteria

AC-1 • Coverage updates incrementally as evidence lands

- **Given** an active meeting
- **When** evidence attaches to a question
- **Then** the coverage fractions for WF1, WF2 and WF3 are recomputed incrementally and pushed as a **coverage** frame per Design §10.2.3
- **And** the recomputation does not require a full re-analysis of the transcript

AC-2 • The three workflows are shown separately and honestly

- **Given** a meeting that has covered brand discovery thoroughly and collected no creative assets
- **When** the operator looks at the coverage display
- **Then** WF1 reads high and WF3 reads low, with the specific missing items nameable on demand — not a single blended percentage
- **And** WF3's requirements include business photos and previous ads, per Design §1.1 goal G-2

AC-3 • The gap is actionable, not decorative

- **Given** a low WF3 coverage reading
- **When** the operator opens it
- **Then** they see the specific unmet items — "no business photos captured", "no previous ads provided" — each linking to the action that would resolve it, such as opening the capture flow from H-01

AC-4 • Coverage is what gates the process button

- **Given** the meeting ending
- **When** coverage is below the configured threshold for any workflow
- **Then** the session can still close, but the state carries the shortfall so J-01's process button can warn rather than silently proceeding
- **And** the operator is never blocked from ending a meeting by a coverage score

Technical notes

- SKL-OIA-09 runs in two modes: incremental during LIVE and full during PROCESS. Same skill, same prompt, different input window. Build the incremental path here and reuse it in J-01 rather than writing a second coverage implementation.
- AC-2 is the direct expression of the requirement that onboarding is not limited to the five-page wizard. A single blended number would hide exactly the failure mode this feature exists to prevent — a meeting that fills the wizard beautifully and leaves WF3 with nothing to work from.
- EVT-107 carries the fractions and the delta that caused the update. The delta is what makes the number debuggable when an operator disputes it.

Test cases

File	Case	Proves
tests/test_session_state.py	test_coverage_incremental_matches_full	The two modes agree
tests/test_session_state.py	test_wf3_low_without_assets	AC-2's core case
frontend/__tests__/CoveragePanel.test.tsx	test_gap_items_link_to_actions	Actionability
tests/e2e/test_live_meeting.py	test_meeting_closes_below_threshold	Never blocking

11. Epic H — Document Capture and OCR

The brand owner puts a document, a product photo or an old printed ad in front of the camera and the system reads it. Design §8.4 specifies the pipeline in full; this epic implements it in four slices — stills, snippets, the image OCR path, and the video OCR path.

One design decision drives the acceptance criteria throughout: **the low-confidence warning must reach the operator while the document is still on the table**. Design §8.4 states it plainly — catching a bad read after the meeting is worthless. That is why H-03's latency criterion exists and why the retake prompt is in the meeting view rather than in the review page.

H-01 • Photo capture with usage tagging

3 pts • Full-stack • Depends on E-02, B-02 • Blocks H-02, H-03 • Design §10.1 [BrandAsset](#), §11 [CaptureControl](#) • Requirements FR-CAP-01, FR-CAP-05

As an Admin, I want to photograph a document or product through the meeting view and tag what it is for, so that the material the brand owner brought to the meeting reaches the workflows that need it.

Acceptance criteria

AC-1 • Capture works without leaving the meeting

- **Given** an active meeting
- **When** the operator taps the capture icon
- **Then** the camera preview opens as an overlay, a still can be taken and confirmed or retaken, and dismissing it returns to the meeting with recording and transcription uninterrupted

AC-2 • Every capture is tagged before it is stored

- **Given** a confirmed still
- **When** it is saved
- **Then** the operator selects a [usage_tag](#) from [business_photo](#), [previous_ad](#), [identity_document](#), [brand_asset](#) or other
- **And** the tag defaults to nothing — there is no "most common" default, because a mis-defaulted identity document is a data-protection incident

AC-3 • The capture is registered as a first-class asset

- **Given** a tagged capture
- **When** it is uploaded
- **Then** it follows the existing [_landing/{tenant_id}/{uuid}_{filename}](#) path, is registered as a [BrandAsset](#) with [onboarding_session](#) set and [usage_tag](#) populated, and appears in the right-hand rail immediately
- **And** the upload is idempotent under retry

AC-4 • Captures survive a bad network like recordings do

- **Given** a capture taken while the network is down
- **When** connectivity returns
- **Then** the capture uploads from the local queue with its tag intact
- **And** the operator is never asked to re-photograph a document the brand owner has already put away

Technical notes

- Reuse F-03's queue-and-drain machinery rather than writing a second upload path. The failure mode in AC-4 is identical and the operator cost of getting it wrong is higher — audio can be re-recorded from the spool, a document cannot be re-photographed once it is back in a bag.
- `identity_document` is the tag that triggers PG-08's restrictions downstream. Make the selection list order put it away from the likely-tapped position; a fat-finger mis-tag in the other direction is far cheaper than in this one.
- Do not compress aggressively client-side. H-03's OCR accuracy depends on it, and bandwidth is not the binding constraint in a meeting room.

Test cases

File	Case	Proves
<code>tests/test_backend_contracts.py</code>	<code>test_capture_registers_brand_as_set</code>	The asset contract
<code>tests/test_idempotency.py</code>	<code>test_capture_upload_idempotent</code>	Retry safety
<code>frontend/__tests__/CaptureControl.test.tsx</code>	<code>test_no_default_usage_tag</code>	AC-2's second clause
<code>frontend/__tests__/CaptureControl.test.tsx</code>	<code>test_capture_does_not_pause_recording</code>	AC-1

H-02 • Short video snippet capture

2 pts • Full-stack • Depends on H-01 • Blocks H-04 • Design §8.4 video path • Requirements FR-CAP-02

As an Admin, I **want** to record a short video snippet of a multi-page or physical item, **so that** something that cannot be captured in one still is still captured.

Acceptance criteria

AC-1 • Snippets are bounded

- **Given** the snippet recorder
- **When** recording runs past 30 seconds
- **Then** it stops automatically, with a visible countdown from 10 seconds so the stop is expected rather than surprising
- **And** the bound is a config value, because H-04's per-frame OCR cost scales directly with it

AC-2 • Snippets carry the same tagging and registration as stills

- **Given** a confirmed snippet
- **When** it is saved
- **Then** it goes through the identical tagging flow and `BrandAsset` registration as H-01, with `modality` distinguishing it
- **And** it appears in the same right-hand rail

AC-3 • Snippet capture does not compete with the meeting recording

- **Given** an active audio recording
- **When** a snippet is captured

- **Then** the meeting's audio recording continues without interruption and the snippet's own audio track is discarded
- **And** the transcript has no gap across the capture window

Technical notes

- AC-3 is the trap. Requesting a video stream can renegotiate the audio device on some browsers and silently interrupt `MediaRecorder`. Acquire the video track without touching the existing audio track, and make the "no gap in transcript" assertion an automated test rather than a manual check.
- Discarding the snippet's audio is deliberate: the meeting microphone is already recording the room, and a second consented-differently audio track is a compliance problem for no benefit.

Test cases

File	Case	Proves
<code>frontend/__tests__/CaptureControl.test.tsx</code>	<code>test_snippet_bounded_at_config_limit</code>	AC-1
<code>tests/e2e/test_live_meeting.py</code>	<code>test_no_transcript_gap_across_snippet</code>	AC-3, the real risk
<code>tests/test_backend_contracts.py</code>	<code>test_snippet_registers_with_modality</code>	Consistent registration

H-03 • Image OCR pipeline

3 pts • Backend • **Depends on** H-01 • **Blocks** H-04, J-02 • **Design** §8.4 image path, §8.1 SKL-OIA-07 • **Requirements** FR-CAP-03, FR-CAP-04

As a System, I want captured stills preprocessed, read by Cloud Vision and interpreted by Gemini, **so that** the text on a document becomes evidence the extraction step can use.

Acceptance criteria

AC-1 • The pipeline runs in the order the design specifies

- **Given** an uploaded still
- **When** SKL-OIA-07 runs
- **Then** it preprocesses (deskew and contrast-normalise), calls Cloud Vision `DOCUMENT_TEXT_DETECTION`, produces confidence-weighted text, then runs the Gemini semantic pass
- **And** it returns `{ocr_text, caption, doc_type, usage_tag, sensitivity_class}` per Design §8.4

AC-2 • Low-confidence reads reach the operator during the meeting

- **Given** a capture whose `ocr_confidence` is below 0.5
- **When** analysis completes
- **Then** the meeting view flags that capture "low read — retake suggested" while the meeting is still live
- **And** the flag arrives within a budget short enough to matter — measured and asserted, not assumed

AC-3 • Sensitive media is restricted at the boundary

- **Given** a capture classified `IDENTITY` or `FINANCIAL`
- **When** it is stored and later used

- **Then** PG-08 applies: it is not passed un-redacted to any extraction prompt, and where redaction is not possible it is excluded from RAG entirely
- **And** the exclusion is recorded on the asset so review can explain why the document is not contributing

AC-4 • Stored text is redacted text

- **Given** OCR output containing a personal name and an account number
- **When** it is written to `BrandAsset.ocr_text`
- **Then** the stored text has been through SKL-OIA-16, and `ocr_confidence` reflects the Vision confidence before redaction
- **And** EVT-106 carries `media_id`, `usage_tag`, `ocr_confidence` and `sensitivity_class`, never the text

Technical notes

- `app/providers/ocr.py` holds the Vision client; the Gemini semantic pass goes through `app/providers/llm.py`. Keeping them separate is what makes the `vision`-breaker degradation to `GEMINI_ONLY_OCR` a config-driven path rather than an if-branch through the middle of the skill.
- AC-2's budget is the whole reason this is a live-path story rather than a PROCESS-path story. Pick a number the team can hold — the design's position is simply that the operator must still be in the room — and make the assertion concrete.
- Both providers down means the media is stored and OCR is deferred to a retry queue with exponential backoff, per §8.4. The meeting is never blocked. Build the retry queue here; J-02 will depend on it having drained.

Test cases

File	Case	Proves
<code>tests/test_ocr_pipeline.py</code>	<code>test_pipeline_stage_order</code>	AC-1
<code>tests/test_ocr_pipeline.py</code>	<code>test_low_confidence_flags_during_meeting</code>	The design's own point
<code>tests/test_guardrails.py</code>	<code>test_pg08_blocks_identity_media_in_prompt</code>	The sensitive-media rule
<code>tests/test_ocr_pipeline.py</code>	<code>test_ocr_text_stored_redacted</code>	AC-4
<code>tests/test_circuit_breakers.py</code>	<code>test_vision_open_falls_back_to_gemini</code>	Degradation

Risk. Real documents are photographed at an angle, in bad light, on a table with a patterned surface. Build the fixture set from photographs taken that way, not from clean scans, or AC-2's threshold will be tuned against the wrong distribution.

H-04 • Video snippet OCR

3 pts • Backend • Depends on H-02, H-03 • Blocks J-02 • Design §8.4 video path • Requirements FR-CAP-03

As a System, I want video snippets reduced to keyframes and read the same way stills are, so that a multi-page document filmed in one pass yields the same evidence as photographing every page.

Acceptance criteria

AC-1 • Keyframes come from both a fixed rate and scene changes

- **Given** a 25-second snippet panning across three pages
- **When** extraction runs
- **Then** ffmpeg produces frames at 1 fps plus scene-change detections, so a fast page turn between fixed-rate samples is still captured

AC-2 • Near-duplicate frames are collapsed before OCR spend

- **Given** extracted frames of which many show the same page
- **When** deduplication runs
- **Then** near-duplicates are collapsed by shingling per Design §8.4, and only distinct frames proceed to Vision
- **And** the reduction ratio is logged, because it is the story's cost control and needs to be visible when a snippet costs more than expected

AC-3 • Merged text keeps its frame timestamps

- **Given** per-frame OCR results
- **When** they are merged
- **Then** the merged `ocr_text` retains frame timestamps, so J-02's provenance can cite `media_id` plus a frame timestamp exactly as §8.4 requires
- **And** the merge does not duplicate a line that appeared in two retained frames

AC-4 • The semantic pass reads the whole document, not one frame

- **Given** merged text spanning three pages
- **When** Gemini runs
- **Then** it receives the merged text and the best representative frames together, and produces one `doc_type` and one caption for the document rather than three disconnected ones

Technical notes

- Verify ffmpeg is present in the service image before this story starts. It is the one dependency in this design that is not a Python package, and discovering it is missing during a Railway deploy is a bad day.
- The dedup threshold is a tuning knob with a direct cost consequence. Expose it in config and log the ratio per AC-2 so it can be tuned from production data rather than from the fixture set.
- Cap the number of frames sent to Vision per snippet. A pathological snippet — a hand-held camera in a shaking room — defeats scene-change dedup entirely, and the cap is what stops one bad capture from costing what a hundred good ones do.

Test cases

File	Case	Proves
tests/test_ocr_pipeline.py	test_scene_change_catches_fast_page_turn	AC-1
tests/test_ocr_pipeline.py	test_dedup_collapses_near_duplicates	Cost control
tests/test_ocr_pipeline.py	test_merged_text_retains_frame_timestamps	Provenance
tests/test_ocr_pipeline.py	test_frame_cap_enforced	The pathological case

12. Epic I — Recordings Library

Three stories that turn a stored recording into something an operator will actually open. The value here is not storage — F-03 already stored it — but retrieval: a summary with timestamps that lets someone find the thirty seconds they need out of fifty minutes.

I-01 • Recordings and captures rail

2 pts • Full-stack • **Depends on** F-03, H-01, E-02 • **Blocks** I-02, K-04 • **Design** §11 [RecordingsRail](#) • **Requirements** FR-LIB-01

As an Admin, I **want** every recording and capture for a session listed in one place, **so that** I can see what the meeting produced without hunting through storage.

Acceptance criteria

AC-1 • The rail lists both recordings and captures, newest first

- **Given** a session with three recordings and six captures
- **When** the rail renders
- **Then** recordings appear newest-first with duration and status, captures appear as thumbnails with their [usage_tag](#), and both are visible without switching tabs

AC-2 • Status is truthful mid-flight

- **Given** a recording still uploading and a capture still being OCR'd
- **When** the rail renders
- **Then** each shows its actual state — uploading, processing, ready — rather than appearing complete
- **And** states advance without a manual refresh

AC-3 • The rail is tenant-scoped and role-aware

- **Given** a Viewer
- **When** the rail renders
- **Then** they can see and play recordings but cannot delete them
- **And** a request for a recording belonging to another tenant returns 404, not 403

Technical notes

- Signed URLs for playback should be short-lived and minted per request. Do not store a playback URL on the model; the asset path is the durable thing.
- AC-2's live advancement can ride the existing WebSocket during a meeting and fall back to polling on the session detail page afterwards. Two mechanisms is acceptable here; one mechanism that holds a socket open on a page nobody is watching is not.

Test cases

File	Case	Proves
tests/test_rbac.py	test_viewer_cannot_delete_recording	Role enforcement
tests/test_rbac.py	test_cross_tenant_recording_404	The isolation convention
frontend/__tests__/RecordingsRail.test.tsx	test_processing_state_visible	AC-2

I-02 • Player and recording summary with key moments

3 pts • Full-stack • Depends on I-01, F-05 • Blocks I-03, K-01 • Design §8.1 SKL-OIA-08, §10.1
[MeetingRecording.summary](#) • Requirements FR-LIB-02

As an Admin, I **want** to open a recording, read its summary, and jump straight to a key moment, **so that** finding what was said takes seconds rather than a replay.

Acceptance criteria

AC-1 • The summary is generated once and stored

- **Given** a recording that has finished uploading
- **When** SKL-OIA-08 runs
- **Then** [MeetingRecording.summary](#) is populated as `{text, key_moments: [{t, label}]}` per Design §10.1
- **And** re-running for the same recording with the same [Idempotency-Key](#) does not produce a second summary

AC-2 • Key moments are navigable, not decorative

- **Given** a summary with key moments
- **When** the operator clicks one
- **Then** the player seeks to that timestamp and plays
- **And** the timestamps are accurate against the audio — verified against a fixture recording with known content at known times, not eyeballed

AC-3 • The player does what a player should

- **Given** the zoom view
- **When** the operator uses it
- **Then** play, pause, seek by scrubbing, and keyboard seek all work, and the current position is reflected in the transcript view when I-03 lands

AC-4 • Summaries reflect redacted content and say so

- **Given** a recording whose transcript had redactions
- **When** the summary renders
- **Then** it is generated from the redacted transcript, and where a redaction removed something material the summary indicates that a redacted value sat there rather than silently omitting it

Technical notes

- Generate the summary on the recording-complete path from F-03, not lazily on first view. An operator opening a recordings list should never wait on a model call.
- Key moments should be labelled in the operator's language — "founding story", "budget discussion" — not with timestamps repeated as labels. The label is the entire retrieval affordance.
- AC-4 matters more than it looks. An operator who reads a summary that omits a redacted phone number and concludes the brand owner never gave contact details has been misled by the system's privacy behaviour.

Test cases

File	Case	Proves
tests/test_idempotency.py	test_summary_generation_idempotent	AC-1
tests/test_backend_contracts.py	test_key_moment_timestamps_accuracy	AC-2
frontend/__tests__/RecordingPlayer.test.tsx	test_key_moment_click_seeks	Navigability
tests/e2e/test_process_to_review.py	test_summary_built_from_redacted_transcript	AC-4

I-03 • Full transcript view

3 pts • Full-stack • Depends on I-02 • Blocks K-01 • Design §11 [TranscriptView](#) • Requirements FR-LIB-03

As an Admin, I **want** the full transcript with speakers and timestamps, synchronised to the player, **so that** I can read what was actually said and cite it exactly.

Acceptance criteria

AC-1 • The transcript is speaker-attributed and timestamped

- **Given** a completed recording
- **When** the transcript opens
- **Then** each segment shows its speaker, its timestamp and its text, with the operator and the brand owner visually distinguished

AC-2 • Transcript and player are two views of one position

- **Given** the player at 12:04
- **When** the transcript is open
- **Then** the corresponding segment is highlighted and kept in view
- **And** clicking a segment seeks the player to it
- **And** manual scrolling in the transcript suspends auto-follow until the operator returns to the current position, per E-02's focus rule

AC-3 • Search works on a long transcript

- **Given** a fifty-minute transcript
- **When** the operator searches
- **Then** matches are found, counted, navigable, and each jumps the player to the match's timestamp
- **And** the view remains responsive at that length, which is a real constraint at several thousand segments

AC-4 • What is shown is what is stored

- **Given** a transcript with redactions
- **When** it renders
- **Then** redacted spans show their typed placeholder — `[PHONE_NUMBER]` — rather than a blank or the original value
- **And** there is no path in this view, for any role, that reveals the pre-redaction text

Technical notes

- Virtualise the list. AC-3's responsiveness constraint is not theoretical at fifty minutes of two-speaker dialogue.
- AC-4 closes the loop opened in F-05: the operator saw the unredacted text live because they were in the room. Afterwards, nobody sees it, including them. That asymmetry is intentional and should be stated in the UI once, not left for someone to discover and file as a bug.
- This view is the citation surface for K-01's review page. Deep-linking to a segment by `recording_id` plus `t_start` should work from the start, because that is exactly the shape of the evidence spans G-02 has been writing all along.

Test cases

File	Case	Proves
frontend/__tests__/TranscriptView.test.tsx	test_autofollow_suspends_on_manual_scroll	The focus rule
frontend/__tests__/TranscriptView.test.tsx	test_search_navigates_and_seeks	AC-3
tests/test_rbac.py	test_no_role_can_read_unredacted_transcript	AC-4's hard claim
tests/e2e/test_process_to_review.py	test_evidence_span_deep_link_resolves	The citation path

13. Epic J — Processing and Auto-Fill

PROCESS is where the meeting becomes data. Design §9.3 specifies it as a job rather than a request, with a 202-plus-callback shape and an **Idempotency-Key** of `sha256(session_id + evidence_manifest_hash)` — so a double-click returns the original job and an added recording produces a genuine re-run.

One rule dominates this epic and it is worth stating before the first card: **a value without evidence is dropped, not written with a caveat**. That is OG-01, and it is the reason the callback carries `dropped_ungrounded` as a first-class number rather than hiding it. The agent may not invent onboarding data. Everything in J-03 and J-04 exists to make that enforceable rather than aspirational.

J-01 • Process dispatch and job lifecycle

2 pts • Full-stack • **Depends on** G-06, B-04 • **Blocks** J-02, K-01 • **Design** §9.3, §10.2.2 • **Requirements** FR-PROC-01

As an Admin, I **want** one button that processes everything the meeting produced, **so that** the gap between finishing a meeting and having a filled-in wizard is one click and a coffee.

Acceptance criteria

AC-1 • The button is available exactly when it should be

- **Given** a session in `GATHERED`
- **When** the operator views it
- **Then** the process button is prominent and enabled
- **And** in any earlier state it is present but disabled with the reason named — "End the meeting to process" — rather than hidden

AC-2 • Coverage shortfall warns, never blocks

- **Given** a session whose WF3 coverage is below threshold
- **When** the operator presses process
- **Then** they see what is thin and what that will cost downstream, with the options to continue or return to capture more
- **And** choosing continue proceeds normally — G-06's AC-4 established that coverage never blocks

AC-3 • Dispatch follows the 202-plus-callback contract

- **Given** a valid dispatch
- **When** Django calls `POST /v1/process`
- **Then** the request carries `X-Service-Token`, the **Idempotency-Key** computed as `sha256(session_id + evidence_manifest_hash)`, and the `evidence_manifest` shape from Design §10.2.2
- **And** the agent replies 202 with `job_id`, `status: ACCEPTED`, `estimated_duration_s` and `callback_url`

AC-4 • Re-running is safe and meaningful

- **Given** a completed job
- **When** process is pressed again with unchanged evidence

- **Then** the original `job_id` is returned and nothing is written twice
- **And** when a recording has been added since, the manifest hash differs and a genuine re-run occurs

AC-5 • Progress is visible

- **Given** a running job
- **When** the operator watches
- **Then** they see stage-level progress — coverage, then page 1 through 5 — not an indeterminate spinner for three minutes
- **And** the session sits in `PROCESSING` throughout, moving to `REVIEW_PENDING` on the terminal callback

Technical notes

- The manifest hash must be computed over a canonical serialisation. A dict ordering difference between two Django processes would silently defeat AC-4's first clause and produce duplicate work that looks like a race condition.
- `estimated_duration_s` should be derived from total audio duration, not a constant. The operator's decision to wait or come back later depends on it being roughly true.

Test cases

File	Case	Proves
tests/test_idempotency.py	test_same_manifest_returns_original_job	AC-4
tests/test_idempotency.py	test_added_recording_triggers_real_rerun	AC-4's second clause
tests/test_backend_contracts.py	test_dispatch_matches_10_2_2_shape	Contract conformance
tests/e2e/test_process_to_review.py	test_state_moves_gathered_to_review	Lifecycle

J-02 • Evidence assembly and full coverage assessment

5 pts • Backend • **Depends on** J-01, G-03, G-05, H-03, H-04 • **Blocks** J-03 • **Design** §9.3, §8.1 SKL-OIA-09, §6 memory • **Requirements** FR-PROC-02

As a System, I want every piece of evidence the session produced gathered into one working context, **so that** extraction reasons over the whole meeting rather than over whatever fits in one prompt.

Acceptance criteria

AC-1 • All evidence types are assembled

- **Given** a session with recordings, transcripts, captured media with OCR text, prepared questions and ad-hoc questions
- **When** assembly runs
- **Then** all of it is retrieved — transcripts from GCS, media from `BrandAsset`, questions with their evidence spans, plus RAG-retrieved indexed chunks
- **And** a capture whose OCR is still in H-03's retry queue is either awaited within a bounded window or explicitly recorded as missing in the job summary, never silently omitted

AC-2 • Context that exceeds the window is compressed, not truncated

- **Given** evidence exceeding 0.75 of the context window
- **When** L2 assembly runs
- **Then** hierarchical summarization compresses older material while preserving evidence references intact
- **And** an evidence span that survives compression still resolves to (`recording_id`, `t_start`, `t_end`) — losing the span is worse than losing the text, because J-04 will drop anything unreferenced

AC-3 • Full coverage assessment agrees with the live one

- **Given** the same evidence
- **When** SKL-OIA-09 runs in full mode
- **Then** its WF1, WF2 and WF3 fractions are within tolerance of G-06's final incremental values
- **And** where they differ materially, the difference is logged with cause, because a divergence means one of the two paths is wrong

AC-4 • Manual checkmarks are not treated as evidence

- **Given** questions checked manually during a degraded-STT meeting
- **When** assembly runs
- **Then** those questions contribute no extracted values, because they carry no evidence spans
- **And** the coverage assessment reflects that the question was marked answered but the answer was not captured

Technical notes

- This is a 5-point story because AC-2 is genuinely hard. The naive implementation summarises transcript text and loses the spans, which passes every test that checks output quality and fails the one that matters. Summarise into a structure that carries spans, not into prose.
- AC-3 is a cheap and unusually valuable consistency check: two implementations of the same measure, one incremental and one batch, cross-validating each other on every job.
- The RAG retrieval must carry the `tenant_id` metadata filter. It is the existing platform pattern; use the existing helper rather than composing a query by hand.

Test cases

File	Case	Proves
<code>tests/test_memory_compression.py</code>	<code>test_spans_survive_summarization</code>	AC-2, the hard part
<code>tests/test_memory_compression.py</code>	<code>test_compression_at_075_threshold</code>	The configured trigger
<code>tests/test_session_state.py</code>	<code>test_full_coverage_matches_incremental</code>	Cross-validation
<code>tests/e2e/test_degraded_stt.py</code>	<code>test_manual_checks_yield_no_values</code>	AC-4

J-03 • Field extraction and mapping, page by page

5 pts • Backend • **Depends on** J-02, B-03 • **Blocks** J-04, J-05 • **Design** §9.3 loop, §8.2 SKL-OIA-10 • **Requirements** FR-PROC-03

As a System, I want the wizard's fields extracted from the evidence one page at a time with references attached, **so that** the operator opens page 1 already filled in and can see where every value came from.

Acceptance criteria

AC-1 • Extraction runs per page with a bounded step budget

- **Given** assembled evidence
- **When** extraction runs
- **Then** SKL-OIA-10 is invoked once per wizard page, PG-01's plan is emitted first and PG-02's 40-step budget is enforced
- **And** exceeding the budget terminates the job with a typed error rather than looping

AC-2 • Every candidate carries evidence references

- **Given** an extracted value
- **When** it is returned
- **Then** it carries (`recording_id`, `t_start`, `t_end`) or `media_id` plus frame timestamp
- **And** the reference resolves to real content — a reference to a span that does not exist is a failure, not a warning

AC-3 • The 13 new Company fields are extracted, not just the original wizard set

- **Given** evidence covering competitors, sales channels and the founder story
- **When** extraction runs
- **Then** the new fields from B-03 are populated alongside the original ones
- **And** WF3-relevant material — the captures tagged `previous_ad`, the business photos — is mapped through to the creative-relevant fields rather than being left as unattached assets

AC-4 • Manually set fields are never overwritten

- **Given** a field whose `FieldProvenance.status` is `EDITED` or `CONFIRMED`
- **When** a re-run produces a different value
- **Then** PG-06 blocks the overwrite and the difference routes to J-05 as a conflict
- **And** this holds on the bulk write path, not only on the per-field path

AC-5 • Output conforms to schema

- **Given** any extraction result
- **When** it is parsed
- **Then** it validates against the declared JSON schema, and a malformed result is retried once then dropped with `ERR` logging rather than partially applied

Technical notes

- Page-at-a-time is not just a context-size tactic; it is what makes AC-5's failure isolated. One page failing schema validation should not lose the other four pages' work.
- `apps/onboarding/field_map.py` is the single source for field targets and is shared with C-03 and G-05. If extraction resolves a field name this map does not know, that is a bug in the map, not a reason to special-case here.
- AC-4's second clause is the one that gets missed. `bulk_create` and `bulk_update` bypass `save()`, exactly as B-05 warned; enforce PG-06 in the write path itself.

Test cases

File	Case	Proves
tests/test_guardrails.py	test_pg06_blocks_bulk_overwrite	AC-4's real risk
tests/test_backend_contracts.py	test_new_company_fields_extracted	AC-3
tests/test_backend_contracts.py	test_schema_violation_isolated_to_page	AC-5
tests/e2e/test_process_to_review.py	test_all_five_pages_prefilled	The user-visible promise

J-04 • Grounding enforcement and KEY/SECONDARY classification

3 pts • Backend • Depends on J-03 • Blocks K-01, K-02 • Design §5.3 OG-01/OG-02/OG-03 • Requirements FR-PROC-04, NFR-SEC-04

As a System, I want ungrounded values dropped and uncertain values forced into explicit review, **so that** nothing reaches a client's brand strategy that the agent invented.

Acceptance criteria**AC-1 • OG-01 drops rather than caveats**

- **Given** an extracted value with no resolvable evidence reference
- **When** OG-01 evaluates it
- **Then** it is dropped and logged, and the drop count increments **dropped_ungrounded**
- **And** it is not written with a low confidence, not written with a "needs review" flag, and not written at all

AC-2 • The drop count is surfaced, not buried

- **Given** a job that dropped six values
- **When** the callback returns
- **Then** **dropped_ungrounded: 6** is in the summary and is displayed in the review UI
- **And** it appears on the observability dashboard, because a rising trend is the earliest signal of a prompt regression

AC-3 • Low confidence forces KEY classification

- **Given** an extracted value with confidence below 0.6
- **When** OG-03 evaluates it
- **Then** the field is classified **KEY** regardless of what it would otherwise have been, guaranteeing explicit human review
- **And** the agent never writes a value it is quietly unsure about as a **SECONDARY** field

AC-4 • Egress redaction is applied again

- **Given** an extracted value that a model re-emitted from inferred context — a name it reconstructed rather than read
- **When** OG-02 evaluates the output
- **Then** redaction is re-applied before UI delivery and before golden-dataset capture

- **And** the belt-and-braces relationship with IG-04 is tested explicitly, with a fixture where IG-04 alone would have been insufficient

AC-5 • Tenant isolation is checked at egress

- **Given** any output payload
- **When** OG-05 evaluates it
- **Then** a cross-tenant identifier raises a security event, blocks the response, and alerts — it is not logged as a data-quality warning

Technical notes

- OG-01's implementation should resolve the reference, not merely check that a reference field is non-empty. A hallucinated span with plausible timestamps passes a presence check and fails a resolution check, and the second is the one worth having.
- The 0.6 threshold in OG-03 and the 0.7 threshold in G-03 are different numbers for different decisions. Name them separately in config; collapsing them is a tempting simplification that couples a live UX knob to a data-safety floor.

Test cases

File	Case	Proves
tests/test_guardrails.py	test_og01_resolves_not_just_presence	The real check
tests/test_guardrails.py	test_og03_forces_key_below_06	AC-3
tests/test_guardrails.py	test_og02_catches_reemitted_entity	Belt and braces
tests/test_guardrails.py	test_og05_cross_tenant_is_security_event	AC-5

J-05 • Conflict detection and escalation

2 pts • Backend • **Depends on** J-03, B-05 • **Blocks** K-02 • **Design** §8.3 SKL-OIA-14, §13.1
[agent.escalations](#) • **Requirements** FR-PROC-05

As an Admin, **I want** contradictions surfaced as decisions rather than resolved behind my back, **so that** when the brand owner said two different things I am the one who picks.

Acceptance criteria

AC-1 • Conflicting values become an explicit conflict record

- **Given** two evidence-backed values for the same field
- **When** SKL-OIA-14 runs
- **Then** a `FieldProvenance` row with `CONFLICT` status is created carrying both candidates and both evidence references
- **And** neither value is written to `Company` — the field stays empty pending a decision

AC-2 • Escalation reaches the shared queue

- **Given** a conflict
- **When** it is raised

- **Then** EVT-007 `agent.escalated` is emitted and a message lands on `agent.escalations` with its 30-day retention
- **And** the payload carries evidence references, never evidence text

AC-3 • A re-run conflict against a confirmed value behaves differently

- **Given** a field an Admin has already `CONFIRMED`
- **When** a re-run extracts a different value
- **Then** PG-06 preserves the confirmed value and the new candidate is raised as a conflict for review
- **And** the confirmed value remains live in the meantime — the client's data does not become empty because the agent changed its mind

AC-4 • The count is reported

- **Given** a job with one conflict
- **When** the callback returns
- **Then** `conflicts: 1` appears in the summary alongside `fields_written` and `dropped_ungrounded`

Technical notes

- AC-1's "neither value is written" is the deliberate choice. Writing the higher-confidence one and flagging it means the flag gets dismissed and the wrong value ships. An empty field with a visible decision gets resolved.
- Conflicts are common and benign in practice — a brand owner correcting themselves mid-meeting is a conflict. Rank the candidates by recency of evidence so the review UI can offer a sensible default without deciding for the operator.

Test cases

File	Case	Proves
<code>tests/test_backend_contracts.py</code>	<code>test_conflict_writes_neither_value</code>	AC-1
<code>tests/test_kafka_roundtrip.py</code>	<code>test_escalation_carries_refs_no_text</code>	Payload hygiene
<code>tests/test_guardrails.py</code>	<code>test_confirmed_value_survives_rerun</code>	AC-3

J-06 • Auto-generation of brand strategy and identity

2 pts • Backend • Depends on J-04 • Blocks K-01 • Design §8.2 SKL-OIA-12, §2.2 downstream • Requirements FR-PROC-06

As an Admin, I want brand strategy and identity generated automatically once onboarding data is written, so that I have something to react to rather than a blank workflow.

Acceptance criteria

AC-1 • Generation is triggered through the existing endpoints

- **Given** a completed extraction with `options.auto_generate_strategy` and `auto_generate_identity` true
- **When** SKL-OIA-12 runs

- **Then** it calls Django's existing `generate_brand_strategy` and `generate_brand_identity` endpoints
- **And** it does not call the WF2 agents directly — the orchestration those endpoints already perform is not reimplemented here

AC-2 • Generation failure does not fail the job

- **Given** a WF2 agent that errors
- **When** generation fails
- **Then** the PROCESS job still reports `SUCCEEDED` with `generated: []` and a named reason, because the onboarding data — the thing the operator actually needs — was written successfully
- **And** the operator can retrigger generation from the workflow UI

AC-3 • Regeneration is an operator action, not an automatic retry

- **Given** generated output the Admin is unhappy with
- **When** they choose to regenerate
- **Then** the existing manual trigger produces new output from the same onboarding data
- **And** the agent does not automatically regenerate on its own judgement

Technical notes

- AC-2 is the risk-containment decision in this story. Coupling onboarding success to WF2 availability would mean a WF2 outage looks to the operator like a failed meeting.
- AC-3 encodes the answer already agreed during requirements: auto-generate, show the Admin, and let them retrigger if unhappy.

Test cases

File	Case	Proves
<code>tests/test_backend_contracts.py</code>	<code>test_uses_existing_generate_endpoints</code>	AC-1
<code>tests/test_circuit_breakers.py</code>	<code>test_wf2_failure_does_not_fail_process</code>	AC-2
<code>tests/e2e/test_process_to_review.py</code>	<code>test_strategy_and_identity_generated</code>	The happy path

14. Epic K — Review, Wizard Extension and PDF

The operator's last mile. Design §10.3 keeps PDF generation in Django — decision D-09 — because the renderer already works, already handles tenant GCS pathing, and already participates in the RAG upsert. This epic extends what exists rather than replacing it, which is also the shape of the compatibility promise: **the five-page wizard still works with no meeting at all, and this epic must not change that.**

K-01 • Review page with key findings and provenance

3 pts • Full-stack • Depends on J-04, J-06, I-03 • Blocks K-02 • Design §11 [ReviewPage](#), §10.2.2 summary • Requirements FR-REV-01

As an Admin, I **want** one page summarising what the meeting produced and what the agent concluded, **so that** I can review a meeting's output without opening five wizard pages and three recordings.

Acceptance criteria

AC-1 • The page shows findings, sources and the honest numbers

- **Given** a session in [REVIEW_PENDING](#)
- **When** the review page loads
- **Then** it shows the recording summaries, the KEY fields with their extracted values, the SECONDARY fields grouped and collapsed, and the job summary numbers including [dropped_ungrounded](#) and [conflicts](#)

AC-2 • Every value traces to its source in one click

- **Given** any extracted value
- **When** the operator clicks its provenance indicator
- **Then** they land on the exact transcript segment or captured media it came from, using I-03's deep link
- **And** a media-sourced value shows the capture and its frame timestamp

AC-3 • Coverage is restated at review time

- **Given** a session that ended with thin WF3 coverage
- **When** the review page renders
- **Then** the shortfall is visible with its consequences named — what WF3 will have less to work from
- **And** the operator can act on it by scheduling a follow-up rather than only being informed

AC-4 • Conflicts are prominent

- **Given** unresolved conflicts from J-05
- **When** the page renders
- **Then** they appear above the field lists, not inside them, because an unresolved conflict is a blocking decision and a field value is not

Technical notes

- Do not paginate this page. It is the artefact an operator screenshots and sends to a colleague; a single scrollable page is the right shape.
- [dropped_ungrounded](#) needs one line of explanation next to it. Design §10.2.2 is explicit that this number is surfaced rather than hidden, and a bare number with no context will read as an error rather than as the safety property it is.

Test cases

File	Case	Proves
frontend/__tests__/ReviewPage.test.tsx	test_conflicts_render_above_fields	AC-4
tests/e2e/test_process_to_review.py	test_provenance_click_reaches_segment	AC-2
tests/e2e/test_process_to_review.py	test_dropped_count_visible	The honesty requirement

K-02 • KEY field confirmation and conflict resolution

3 pts • Full-stack • Depends on K-01, B-06, J-05 **• Blocks** K-05, L-02 **• Design** §15 RBAC, §5.3 OG-03 **• Requirements** FR-REV-02, NFR-SEC-05

As an Admin, I **want** to confirm or correct each KEY field and resolve every conflict, **so that** what propagates into brand strategy is something a human asserted.

Acceptance criteria**AC-1 • Confirmation is per field and recorded**

- **Given** a KEY field with an extracted value
- **When** the Admin confirms it
- **Then** `FieldProvenance.status` becomes `CONFIRMED`, `extracted_value` is preserved unchanged, and EVT-109 is emitted with `action: CONFIRM` and `edit_distance: 0`

AC-2 • Editing preserves what was extracted

- **Given** a KEY field the Admin corrects
- **When** they save
- **Then** the new value is written, `status` becomes `EDITED`, and `extracted_value` still holds what the agent produced
- **And** EVT-109 carries the field name, the action and the edit distance — and neither value, per B-06

AC-3 • Only Owner and Admin can confirm

- **Given** an Editor on the review page
- **When** they attempt to confirm a KEY field
- **Then** it is denied per the §15 matrix, EVT-004 is emitted with `rule_id: PG-03`, and the UI does not offer the affordance in the first place
- **And** an Editor can still edit SECONDARY fields

AC-4 • Delegation works where the matrix allows it

- **Given** an Owner who has delegated KEY confirmation to a named Editor for this session
- **When** that Editor confirms
- **Then** it is permitted, recorded against `OnboardingSession.config.key_confirm_delegate`, and audit-logged as a delegated action

AC-5 • Every conflict must be resolved before submit

- **Given** an unresolved conflict
- **When** final submit is attempted
- **Then** it is refused with the specific conflicts named
- **And** resolving one writes the chosen value with `status: CONFIRMED` and its evidence reference retained

Technical notes

- AC-2's preservation of `extracted_value` is not bookkeeping — it is the entire input to L-02's flywheel. A UI that overwrites the extracted value on edit silently removes the service's ability to improve.
- The §15 asymmetry — Editors run every extraction skill but cannot confirm KEY fields — is the reason AC-3 exists as a separate criterion. It is easy to implement RBAC as a single per-page check and lose the distinction.

Test cases

File	Case	Proves
tests/test_rbac.py	test_editor_cannot_confirm_key_field	The §15 asymmetry
tests/test_rbac.py	test_owner_delegate_can_confirm	AC-4
tests/test_backend_contracts.py	test_edit_preserves_extracted_value	The flywheel input
tests/e2e/test_process_to_review.py	test_submit_blocked_on_conflict	AC-5

K-03 • Wizard field extension

2 pts • Full-stack • **Depends on** B-03 • **Blocks** K-05 • **Design** §11, §10.1 **Company** • **Requirements** FR-REV-03, NFR-COMPAT

As an Admin, I **want** the new fields available on the existing wizard pages, **so that** I can fill them by hand exactly as I fill everything else.

Acceptance criteria

AC-1 • New fields appear on the right pages, as ordinary inputs

- **Given** the five wizard pages
- **When** they render
- **Then** the 13 new **Company** fields appear on their designated pages with the same validation, styling and save behaviour as existing fields

AC-2 • The manual path is unchanged

- **Given** a company with no onboarding session at all
- **When** an operator completes the wizard by hand
- **Then** every page works, submit works, and the PDF generates — exactly as before this project began
- **And** this is asserted by an automated end-to-end test, not by inspection

AC-3 • Pre-filled fields are marked without being locked

- **Given** a wizard page reached from a processed session
- **When** it renders

- **Then** agent-filled fields carry a subtle provenance marker and remain fully editable
- **And** editing one from the wizard produces the same **FieldProvenance** transition as editing it from the review page — one write path, two entry points

Technical notes

- AC-3's "one write path" is what stops the two surfaces from drifting. Route the wizard's field save through the same provenance-aware endpoint B-06 built.
- Nothing about the wizard's page structure changes. Design §11 is explicit that the five pages are not restructured; they gain inputs.

Test cases

File	Case	Proves
<code>tests/e2e/test_process_to_review.py</code>	<code>test_manual_wizard_path_intact</code>	NFR-COMPAT
<code>tests/test_backend_contracts.py</code>	<code>test_wizard_edit_writes_provenance</code>	AC-3's shared path

K-04 • Meeting evidence in the wizard's final step

2 pts • Full-stack • Depends on I-01, K-03 • Blocks K-05 • Design §10.3 • Requirements FR-REV-04

As an Admin, I **want** the recordings, transcripts and captured media visible at the final wizard step, **so that** the last thing I see before submitting is the evidence behind what I am submitting.

Acceptance criteria

AC-1 • Evidence is listed at step 5

- **Given** a session with recordings and captures
- **When** step 5 renders
- **Then** the recordings appear with durations and summary links, and the captures appear with their `usage_tag`
- **And** each links back to I-02's player or the capture itself

AC-2 • A wizard with no session shows nothing extra

- **Given** a manually completed wizard with no onboarding session
- **When** step 5 renders
- **Then** the evidence section is absent entirely — not present and empty
- **And** the step looks exactly as it did before this project

AC-3 • Consent is referenced

- **Given** recordings covered by a consent record
- **When** step 5 renders
- **Then** the consent reference is shown — method and date, not the subject's details — so the submitting Admin can see the lawful basis

Technical notes

- AC-2 is the compatibility promise appearing again at a different surface. Conditional rendering on session presence, tested.

Test cases

File	Case	Proves
frontend/__tests__/WizardStep5.test.tsx	test_no_evidence_section_without_session	AC-2
tests/e2e/test_process_to_review.py	test_step5_lists_recordings_and_captures	AC-1

K-05 • PDF extension — meeting evidence and key findings

2 pts • Backend • Depends on K-02, K-04 • Blocks — • Design §10.3, decision D-09 • Requirements FR-REV-05

As an Admin, I **want** the onboarding PDF to include the meeting evidence and key findings, **so that** the artefact we archive and index reflects how the data was actually gathered.

Acceptance criteria

AC-1 • Two new sections, existing sections untouched

- **Given** final submission for a session-backed company
- **When** `generate_onboarding_pdf` runs
- **Then** the PDF contains every existing section unchanged, plus a **Meeting Evidence** section (per-recording summary with key moments, the consent reference, and the captured media list with `usage_tag` and one-line OCR-derived descriptions) and a **Key Findings** section mirroring the review page's KEY fields with their confirmation status

AC-2 • Generation stays in Django

- **Given** the generation path
- **When** it runs
- **Then** it is the existing fpdf2 generator in Django, extended — the agent's only responsibility is that all data is written before the Admin reaches step 5
- **And** no PDF rendering code is added to the agent service

AC-3 • Storage and indexing semantics are preserved

- **Given** a generated PDF
- **When** it is stored
- **Then** it is uploaded to tenant GCS, registered as a `BrandAsset` and RAG-indexed exactly as today, with upsert semantics intact
- **And** regenerating for the same company upserts rather than accumulating duplicates

AC-4 • A manual submission produces the original PDF

- **Given** a company with no onboarding session
- **When** the PDF generates
- **Then** it contains the original sections only, with no empty evidence headings

Technical notes

- Redacted OCR descriptions only. The PDF is RAG-indexed and therefore reachable by retrieval; an unredacted identity document description in it would defeat every guardrail upstream.
- AC-3's upsert behaviour already exists. The test is here to catch a regression introduced by the new sections changing the object name or content hash.

Test cases

File	Case	Proves
tests/test_backend_contracts.py	test_pdf_upsert_not_duplicate	AC-3
tests/e2e/ test_process_to_review.py	test_pdf_has_evidence_and_findings	AC-1
tests/e2e/test_gdpr_erasure.py	test_pdf_ocr_descriptions_redacted	The retrieval hazard

15. Epic L — `prompt-optimization-svc` Integration

Design §17 is the answer to a question asked and settled during requirements: yes, this agent should use POI, and tenant customization is in v1 rather than deferred. Three properties from §17.2 drive this epic — resolution runs once per session and never in the live loop, the hardcoded fallback is unconditional, and the agent has no write path into the registry.

That last one is worth restating because it removes a whole threat class: a bad prompt cannot reach production through this agent, because the capability to write one does not exist here.

L-01 • Prompt resolution and session pinning

3 pts • Backend • **Depends on** A-05, A-06 • **Blocks** L-02, L-04, L-05 • **Design** §17.2, §8.3 SKL-OIA-15 • **Requirements** FR-OPT-01, FR-OPT-02

As a System, I want prompts resolved once per session and pinned for its lifetime, **so that** a canary promotion mid-meeting cannot silently change how the agent scores a conversation.

Acceptance criteria

AC-1 • The four-step resolution chain runs in order

- **Given** a session starting in any mode
- **When** SKL-OIA-15 resolves each required `prompt_id`
- **Then** it tries the tenant variant in Redis DB 2, then the platform default in DB 2, then the POI API with a 15-minute write-through cache, then the hardcoded fallback — in that order, stopping at the first hit

AC-2 • Resolution happens once, outside the live loop

- **Given** a live session
- **When** it runs for 45 minutes
- **Then** resolution occurred exactly once, at connect, and no POI call or DB 2 read happens inside the per-segment analysis loop

AC-3 • Versions are pinned and recorded

- **Given** resolved prompts
- **When** the session starts
- **Then** `OnboardingSession.prompt_versions` holds the `{prompt_id: version}` map per PG-05
- **And** a canary promotion during the session does not change what the session uses

AC-4 • The fallback is unconditional

- **Given** Redis cold, POI unreachable and the network unreliable
- **When** a session starts
- **Then** it starts, using versioned hardcoded prompts from `app/prompts/fallbacks.py`, with a `DEGRADED` outcome on EVT-001
- **And** the operator is not told anything, because per §18.2 the `poi` breaker's `user_message` is deliberately null — this failure is invisible by design

Technical notes

- The hardcoded fallbacks must be versioned strings, not inline literals scattered through skills. A fallback with no version makes the resulting golden-dataset candidates unattributable.
- AC-2 is testable by counting calls. Assert zero POI calls and zero DB 2 reads between the first and last segment of a fixture session; it is a cheap test that catches a costly regression.

Test cases

File	Case	Proves
tests/test_prompt_loading.py	test_resolution_order_four_steps	AC-1
tests/test_prompt_loading.py	test_no_poi_calls_inside_live_loop	AC-2
tests/test_prompt_loading.py	test_starts_with_everything_down	AC-4
tests/test_session_state.py	test_versions_pinned_across_promotion	AC-3

L-02 • Golden-dataset candidate topic and emission

3 pts • Backend • **Depends on** L-01, K-02 • **Blocks** L-03, L-04 • **Design** §13.1, §13.3, §17.3, §8.2 SKL-OIA-13 • **Requirements** FR-OPT-03

As a System, I **want** every admin correction captured as a redacted golden-dataset candidate, **so that** the agent's extraction gets measurably better from the work the operator was already doing.

Acceptance criteria

AC-1 • The new topic exists with the right configuration

- **Given** the Kafka topology
- **When** it is provisioned
- **Then** `onboarding.golden-dataset.candidates` exists keyed on `tenant:prompt_id` with 30-day retention, per Design §13.1
- **And** it is the only new topic this service introduces — the other six come from the existing fleet Terraform module with only the name variable changed

AC-2 • Candidates are emitted on divergence, not on every review

- **Given** an EVT-109 review action
- **When** the final value differs from `extracted_value`
- **Then** SKL-OIA-13 emits a candidate carrying `prompt_id`, `prompt_version`, evidence references and the corrected value
- **And** a confirmation with zero edit distance emits no candidate — there is nothing to learn from agreement at that granularity

AC-3 • Sufficiency pairs are captured too

- **Given** an agent green signal that the operator manually overrode
- **When** the override is recorded
- **Then** a candidate is emitted for `oia.sufficiency` with the binary label
- **And** this is a cleaner label than the extraction case, which is why it is captured separately rather than folded in

AC-4 • Redaction precedes capture

- **Given** a corrected value containing personal data
- **When** the candidate is built
- **Then** OG-02 has already run, and the candidate carries evidence *references* rather than evidence text, per §17.3
- **And** a test asserts no known fixture secret appears anywhere in a serialised candidate

Technical notes

- The key is `tenant:prompt_id`, which is what lets POI's consumer partition per prompt per tenant without a repartition step. Getting the key wrong here is expensive to change later because it changes partition assignment.
- This is the one piece of Kafka work in the whole backlog, per Design §13.1. Everything else is topic-name configuration.

Test cases

File	Case	Proves
tests/test_kafka_roundtrip.py	test_candidate_topic_key_and_retention	AC-1
tests/test_kafka_roundtrip.py	test_no_candidate_on_zero_edit_distance	AC-2
tests/test_kafka_roundtrip.py	test_sufficiency_override_emits_candidate	AC-3
tests/test_events_no_pii.py	test_candidate_has_refs_not_text	AC-4

L-03 • Prompt registration and scorer wiring

2 pts • Backend • Depends on L-01, G-04 • Blocks L-04 • Design §17.1 • Requirements FR-OPT-04

As a System, I want all eight prompts registered in POI with their scorers, so that offline GEPA has something well-defined to optimise against.

Acceptance criteria**AC-1 • Eight prompts registered with the §17.1 identifiers**

- **Given** the POI registry
- **When** registration runs
- **Then** `oia.research_brief`, `oia.generate_questionnaire`, `oia.analyze_stream`, `oia.sufficiency`, `oia.followups`, `oia.media_analysis`, `oia.summarize_recording` and `oia.extract_fields` exist with their §17.1 primary scorers attached

AC-2 • The proxy metrics are wired to real signals

- **Given** the scorers
- **When** they compute
- **Then** `oia.followups` reads EVT-105's `accepted` backfill from G-04, `oia.sufficiency` reads agreement with the admin's final checkbox from G-03, and `oia.extract_fields` reads field-level match against the admin's final value from K-02

- **And** each metric traces to an event that actually exists rather than to a placeholder

AC-3 • Registration is idempotent and versioned

- **Given** a re-run of registration
- **When** it executes
- **Then** existing prompts are not duplicated and versions are not reset

Technical notes

- AC-2 is the story's substance. Registering prompts is trivial; wiring each scorer to a signal that is genuinely produced is what makes the registry useful rather than decorative. Where a signal does not yet exist, register the prompt without that scorer and record the gap — do not register a scorer that will silently return a constant.

Test cases

File	Case	Proves
tests/test_prompt_loading.py	test_all_eight_prompts_registered	AC-1
tests/test_prompt_loading.py	test_registration_idempotent	AC-3
tests/test_kafka_roundtrip.py	test_scorer_signals_have_sources	AC-2

L-04 • Tenant prompt customization

3 pts • Full-stack • **Depends on** L-01, L-02, L-03 • **Blocks** — • **Design** §17.3 tenant customization, OD-3 • **Requirements** FR-OPT-05

As a tenant Owner, I **want** prompts specialised to my industry, **so that** the questions the agent prepares for a coffee roaster are not the questions it prepares for a law firm.

Acceptance criteria

AC-1 • A tenant variant starts as an exact clone

- **Given** a newly provisioned tenant
- **When** variants are scaffolded from the `tenant.agent.provisioned` event
- **Then** each variant is a clone of the platform PRODUCTION prompt
- **And** a tenant that never customises behaves identically to one with no variant at all — asserted by comparing resolved prompt bodies

AC-2 • Customisation follows the normal lifecycle

- **Given** an Owner editing a tenant variant
- **When** they promote it
- **Then** it moves through DRAFT → STAGING → CANARY → PRODUCTION with the existing POI gates unmodified — OPT-03's 5% aggregate improvement, OPT-04's 3% per-scorer regression ceiling, 10% canary over 24 hours, automatic rollback

AC-3 • Tenant GEPA unlocks only above the dataset floor

- **Given** a tenant with fewer than the configured minimum curated examples for a `prompt_id`

- **When** tenant-specific GEPA is requested
- **Then** it is refused with the current count and the threshold stated
- **And** the threshold is configuration, tracked as OD-3 with a proposed value of 50

AC-4 • Resolution prefers the tenant variant

- **Given** a tenant with a PRODUCTION variant of `oia.generate_questionnaire`
- **When** a session starts
- **Then** L-01's step 1 resolves the variant, and the pinned version records that it was a tenant variant, not the platform default

Technical notes

- AC-1 is what makes this feature safe to ship in v1. Scaffolding clones means the customization surface exists without changing anyone's behaviour until they deliberately change it.
- Hand-editing a variant is a legitimate path and probably the common one at first — an industry-specialised questioning prompt written by a human is useful long before there are 50 curated examples to optimise against.

Test cases

File	Case	Proves
tests/test_prompt_loading.py	test_clone_variant_resolves_idempotically	AC-1
tests/test_prompt_loading.py	test_tenant_variant_preferred	AC-4
tests/test_prompt_loading.py	test_gepa_refused_below_floor	AC-3

L-05 • Cache invalidation and fallback hygiene

2 pts • Backend • Depends on L-01 • Blocks — • Design §17.2, §20 prompt incident • Requirements FR-OPT-06

As an operator, I want a documented, tested way to recover from a bad prompt, so that a prompt incident is a five-minute procedure rather than an investigation.

Acceptance criteria

AC-1 • The manual recovery path works as documented

- **Given** a bad prompt in PRODUCTION
- **When** the runbook procedure is followed — set the prior version to PRODUCTION in POI, bust the Redis prompt cache, let sessions pick it up at next start
- **Then** new sessions resolve the reverted version and in-flight sessions are unaffected because their versions are pinned

AC-2 • Cache busting is a supported operation, not a manual Redis command

- **Given** the need to invalidate
- **When** the operator invokes the documented mechanism
- **Then** the `poi:prompt:onboarding-intelligence:*` keys for the affected prompt are cleared without touching other services' keys in DB 2

AC-3 • Fallbacks are exercised regularly, not just written

- **Given** the hardcoded fallback set
- **When** the test suite runs
- **Then** every fallback is exercised against a real skill invocation and produces schema-valid output
- **And** a fallback that has drifted out of schema compatibility fails the build rather than surfacing during an outage

Technical notes

- AC-3 is the story's real value. Hardcoded fallbacks rot silently because nothing exercises them until the day everything else is already broken.
- DB 2 is shared and owned by `prompt-optimization-svc`. Scope every operation to this service's prefix; a wildcard flush would take out other agents' prompt caches.

Test cases

File	Case	Proves
tests/test_prompt_loading.py	test_all_fallbacks_schema_valid	AC-3
tests/ test_redis_key_isolation.py	test_cache_bust_scoped_to_prefix	AC-2
tests/test_session_state.py	test_inflight_session_unaffected_by_revert	AC-1

16. Epic M — Security, GDPR and Operations

Nothing in this epic is new capability; it is the epic that makes the previous twelve safe to run. It is scheduled late because most of it can only be tested once the paths exist, and early enough that it is not the thing standing between the team and a launch date.

M-01 • Complete guardrail suite and enforcement audit

3 pts • Backend • **Depends on** A-06, G-01, J-04 • **Blocks** N-01 • **Design** §5.1, §5.2, §5.3 • **Requirements** NFR-SEC-01

As a System, I want every IG, PG and OG rule implemented, wired and individually tested, so that the guardrail table in the design is a description of the code rather than an aspiration.

Acceptance criteria

AC-1 • Every rule has an implementation and a test

- **Given** the rules IG-01...10, PG-01...08 and OG-01...06
- **When** the audit runs
- **Then** each has a named implementation and at least one test that fires it
- **And** an unimplemented rule fails the build rather than being tracked in a spreadsheet

AC-2 • Budgets are enforced as hard timeouts

- **Given** the non-LLM guardrail paths
- **When** they run
- **Then** input evaluation is bounded at 200 ms and output at 300 ms per §18.3, and a breach fails closed and emits EVT-004
- **And** failing closed means the request is refused, not allowed through with a logged warning

AC-3 • OG-04's sampled judge runs at its configured rate

- **Given** generated follow-ups and summaries
- **When** OG-04 samples
- **Then** roughly 10% are judged, failures emit [ERR-08](#), and the failures accumulate into the weekly review
- **And** the sampling rate is config, because it trades cost against detection latency

AC-4 • Triggers are observable by rule

- **Given** any guardrail firing
- **When** it fires
- **Then** EVT-004 carries [rule_id](#) and the action taken, and the observability dashboard shows trigger rate by [rule_id](#)

Technical notes

- AC-1's build-failing audit is a twenty-line test that enumerates the rule catalogue and asserts a registry entry for each. It is the cheapest possible defence against a rule that exists only in the document.
- The weekly guardrail review in §20 is what makes the thresholds tunable without a deploy. Make sure every threshold reachable by that review is in config, not in code.

Test cases

File	Case	Proves
tests/test_guardrails.py	test_every_rule_id_has_implementation	AC-1
tests/test_guardrails.py	test_budget_breach_fails_closed	AC-2
tests/test_guardrails.py	test_og04_sampling_rate_configurable	AC-3

M-02 • GDPR erasure cascade

3 pts • Full-stack • **Depends on** B-07, I-01, L-02 • **Blocks** N-01 • **Design** §20 GDPR operations • **Requirements** FR-GDPR-04

As a tenant Owner, I **want** an erasure request to remove every trace of a subject across the whole system, **so that** we can answer a data-subject request truthfully.

Acceptance criteria

AC-1 • The cascade reaches every store

- **Given** an erasure request from an Admin or Owner
- **When** it executes
- **Then** it removes recordings, transcripts, captured media, summaries, provenance rows, RAG entries and golden-dataset candidates for that subject within that tenant
- **And** it ends in a logged completion report naming what was removed from each store

AC-2 • The list of stores is derived, not maintained by hand

- **Given** a new store added later
- **When** the cascade runs
- **Then** the store registry drives the cascade, and a store not registered fails the cascade's completeness test
- **And** this is why the test asserts against a registry rather than a hardcoded list

AC-3 • Consent revocation enters the same path

- **Given** a `ConsentRecord.revoked_at` being set
- **When** revocation processes
- **Then** it enters the same cascade with the same completeness guarantees
- **And** any open live session for that subject is closed with 4403 per F-01's AC-4

AC-4 • Golden-dataset candidates are included

- **Given** candidates already emitted to Kafka and consumed by POI
- **When** erasure runs
- **Then** the tenant's stored candidates for that subject are removed at POI
- **And** where a candidate has already contributed to an approved dataset, the cascade records that fact explicitly rather than reporting a clean erasure

Technical notes

- AC-4's second clause is the uncomfortable one and it needs to be answered before launch rather than during an audit. Say what the system does, and make the completion report say it too.

- The cascade is a Django concern — it owns the models and the RAG index. The agent's contribution is not holding a second copy of anything.

Test cases

File	Case	Proves
tests/e2e/test_gdpr_erasure.py	test_cascade_covers_registered_stores	AC-1, AC-2
tests/e2e/test_gdpr_erasure.py	test_revocation_uses_same_cascade	AC-3
tests/e2e/test_gdpr_erasure.py	test_golden_candidates_erased	AC-4

M-03 • Configurable retention and enforcement

2 pts • Backend • Depends on M-02 • Blocks N-01 • Design §20, decision D-02 • Requirements FR-GDPR-02

As a tenant Owner, I want to set how long onboarding evidence is kept, so that our retention policy is ours rather than a platform default.

Acceptance criteria

AC-1 • Retention is per tenant with a sane default

- **Given** tenant configuration
- **When** `retention_days` is unset
- **Then** `OIA_RETENTION_DAYS_DEFAULT` of 365 applies
- **And** an Owner or Admin can change it; an Editor or Viewer cannot, per the §15 matrix

AC-2 • The beat job enforces it

- **Given** evidence older than the tenant's window
- **When** the Celery beat job runs
- **Then** it is removed through the same cascade M-02 built
- **And** the job is idempotent and safe to run twice

AC-3 • Shortening retention is not retroactively destructive without warning

- **Given** an Owner reducing retention from 365 days to 30
- **When** they save
- **Then** they are told how much existing evidence that will delete and when
- **And** the deletion happens on the next scheduled run, not instantly, leaving a window to reverse the decision

Technical notes

- AC-3 exists because the alternative is a support ticket that begins "we lost eight months of onboarding data by changing a dropdown."

Test cases

File	Case	Proves
tests/test_rbac.py	test_editor_cannot_set_retention	The §15 matrix

File	Case	Proves
tests/e2e/test_gdpr_erasure.py	test_beat_job_idempotent	AC-2
frontend/__tests__/RetentionSettings.test.tsx	test_shortening_warns_with_counts	AC-3

M-04 • Observability, dashboards and runbook

3 pts • Backend • Depends on A-03, M-01 • Blocks N-02 • Design §20 • Requirements NFR-OPS-01

As an on-call engineer, I **want** the eight dashboard panels and the documented runbook procedures, **so that** a live-meeting incident at 3pm on a Tuesday is a procedure rather than an improvisation.

Acceptance criteria

AC-1 • Probes distinguish liveness from readiness

- **Given** an instance starting
- **When** probes are called
- **Then** `/health` reports liveness only, and `/ready` additionally probes Redis, Kafka and the STT credential path
- **And** a rolling deploy cannot route traffic to an instance that will fail on first use

AC-2 • Eight panels exist with real data

- **Given** the Grafana board
- **When** it loads
- **Then** it shows active WS sessions, STT partial latency p50/p95, sufficiency latency p95, guardrail trigger rate by `rule_id`, circuit state per dependency, DLQ depth, golden-candidate volume, and `dropped_unrounded` per PROCESS job

AC-3 • The stuck-session procedure is automated, not documented

- **Given** a session with no WS heartbeat for 5 minutes
- **When** the watchdog runs
- **Then** the recording is finalised from the GCS spool, the transcript is completed via batch STT, and the state moves to `GATHERED`
- **And** no evidence is lost and the Admin can resume by starting a new recording on the same session

AC-4 • The runbook covers what actually happens

- **Given** the runbook
- **When** an engineer consults it
- **Then** it covers DLQ handling and replay, the stuck-session case, GDPR operations, the guardrail review cycle and the prompt incident procedure — each with a command or a link, not a paragraph of advice

Technical notes

- The `dropped_unrounded` panel is the closest thing the service has to a quality alarm, per §20. Alert on its rate of change, not on an absolute threshold; the absolute number varies legitimately with meeting quality.
- AC-3 is the one runbook entry worth automating rather than documenting, because it fires often and its manual execution is mechanical.

Test cases

File	Case	Proves
tests/test_backend_contracts.py	test_ready_probes_all_dependencies	AC-1
tests/test_session_state.py	test_stuck_session_watchdog_finalises	AC-3
tests/test_kafka_roundtrip.py	test_dlq_replay_preserves_idempotency_key	Safe replay

M-05 • Rate limits, quotas and the live-session lock

2 pts • Backend • Depends on F-04, A-05 • Blocks N-02 • Design §14, §16, OD-5 • Requirements NFR-SEC-06

As a System, I want per-tenant limits and a single-live-session guarantee, so that one tenant's usage cannot degrade another's meeting.

Acceptance criteria**AC-1 • PREP and WS control frames are throttled per user**

- **Given** a user exceeding 10 PREP turns per minute
- **When** the next request arrives
- **Then** it is rate-limited with EVT-012 emitted, and the WS equivalent closes with 4429
- **And** limits are per tenant per user, counted in Redis DB 27

AC-2 • One live session per company, enforced by lock

- **Given** a live session for a company
- **When** a second is started for the same company
- **Then** it is refused, because the lock is keyed on `company_id` rather than `session_id` per §14
- **And** `max_concurrent_sessions` is tenant-configurable for tenants that need more

AC-3 • The lock cannot strand a company

- **Given** a process holding the lock that crashes
- **When** the heartbeat stops
- **Then** the `SET NX PX` TTL expires and a new session can start
- **And** the TTL is long enough that a brief network blip does not release a live lock

AC-4 • The circuit key stays global

- **Given** the breaker state keys
- **When** they are written
- **Then** `oia:v1:circuit:{dep}` is not tenant-scoped, per the design's deliberate exception
- **And** `tests/test_redis_key_isolation.py` records this as an explicit allowed exception rather than failing on it

Technical notes

- AC-4 looks like a violation of the isolation rule and is not. A dependency being down is a global fact, and per-tenant breakers would require every tenant to independently discover an outage. Encode the exception in the test so the next reader does not "fix" it.

Test cases

File	Case	Proves
tests/ test_redis_key_isolation.py	test_circuit_key_exception_alled	AC-4
tests/test_ws_handshake.py	test_rate_limit_closes_4429	AC-1
tests/test_session_state.py	test_live_lock_keyed_on_company	AC-2
tests/test_session_state.py	test_lock_ttl_releases_after_crash	AC-3

17. Epic N — End-to-End Hardening

Three stories that prove the system works rather than adding to what it does. Design §22 sets the targets: 80% unit coverage, 60% integration, and all critical paths covered end to end.

N-01 • End-to-end suite across all five critical paths

3 pts • Full-stack • Depends on M-01, M-02, M-03 • Blocks N-03 • Design §22 • Requirements NFR-QA-01

As the team, I want the five end-to-end journeys automated and green in CI, so that a change anywhere in the service tells us before a customer does.

Acceptance criteria

AC-1 • All five e2e files run in CI against a realistic stack

- **Given** the suite
- **When** CI runs
- **Then** `test_prep_to_questionnaire.py`, `test_live_meeting.py`, `test_process_to_review.py`, `test_degraded_stt.py` and `test_gdpr_erasure.py` all execute against real Redis, real Kafka and faked external providers

AC-2 • The provider fakes are shared and faithful

- **Given** the STT, Vision and LLM fakes
- **When** tests use them
- **Then** they replay recorded fixture responses with realistic timing rather than returning instantly
- **And** a test that passes only because a fake returned in zero milliseconds is a test that will not catch F-05's latency regressions

AC-3 • Coverage targets are enforced, not reported

- **Given** the coverage configuration
- **When** CI runs
- **Then** unit coverage below 80% or integration below 60% fails the build

AC-4 • The suite is fast enough to be run

- **Given** the full suite
- **When** it runs
- **Then** it completes inside a budget the team will tolerate on every pull request
- **And** anything slower than that budget moves to a nightly job explicitly, rather than by everyone quietly skipping it

Technical notes

- AC-4 is the criterion that determines whether any of this survives month three. A suite nobody runs has negative value, because it creates the belief that something is checked.

Test cases

File	Case	Proves
<code>tests/e2e/</code> (all five)	full journeys	AC-1

File	Case	Proves
tests/test_stt_adapter.py	test_fake_replays_realistic_timing	AC-2
CI configuration	coverage gate	AC-3

N-02 • Latency and load verification against the NFRs

3 pts • Backend • **Depends on** N-01, M-04, M-05 • **Blocks** N-03 • **Design** §18.3, §22 • **Requirements** NFR-PERF-01...03

As the team, **I want** the three performance claims measured under load through the real gateway, **so that** we ship numbers rather than intentions.

Acceptance criteria

AC-1 • The three budgets are measured, not asserted

- **Given** a load scenario
- **When** it runs
- **Then** STT partial latency ≤ 2 s, sufficiency feedback ≤ 5 s p95, and a 60-minute-meeting PROCESS job ≤ 5 min p95 are each measured and reported
- **And** measurement goes through Kong on Railway, not against a local socket

AC-2 • Concurrency is realistic

- **Given** the target concurrent live sessions per instance
- **When** load runs at that level
- **Then** the budgets hold, and the point at which they stop holding is recorded as the known capacity limit

AC-3 • Late signals are dropped under load, not queued

- **Given** the system under enough load to exceed the 5-second budget
- **When** sufficiency signals become late
- **Then** they are dropped per §18.3, and the drop rate is visible on the dashboard
- **And** no backlog accumulates that would deliver stale feedback minutes later

AC-4 • The 45-minute meeting holds up

- **Given** a full-length meeting under load
- **When** it runs to completion
- **Then** the WebSocket survives, F-05's stream rollover happens invisibly, memory does not grow unbounded, and the transcript is complete

Technical notes

- AC-1's "through Kong on Railway" repeats F-05's warning because it is the single most common way a latency number becomes fiction.
- AC-4 is where A-02's spike output gets validated against reality rather than against a two-minute test.

Test cases

File	Case	Proves
Load harness	test_partial_latency_p95_through_kong	NFR-PERF-01
Load harness	test_sufficiency_p95_under_concurrency	NFR-PERF-02
Load harness	test_process_60min_within_5min	NFR-PERF-03
tests/e2e/test_live_meeting.py	test_45_minute_session_stable	AC-4

N-03 • Failure injection and DLQ drill

2 pts • Backend • Depends on N-01, N-02 • Blocks — • Design §18.2, §20 • Requirements NFR-REL-03

As the team, I want every degraded mode exercised deliberately before a customer finds it, so that the failure behaviour we designed is the failure behaviour we have.

Acceptance criteria

AC-1 • Every breaker's degraded mode is exercised

- **Given** the seven dependencies in `config/circuit_breakers.yaml`
- **When** each is failed in turn during a live meeting
- **Then** the configured degraded mode engages, the configured `user_message` is what the operator sees, and the meeting completes
- **And** a dependency whose degraded mode has never been exercised fails the drill checklist

AC-2 • Recovery is verified, not assumed

- **Given** each degraded mode
- **When** the dependency recovers
- **Then** the half-open probe closes the breaker, buffered work drains — the Redis outbox, the local pool, the OCR retry queue — and no duplicates result

AC-3 • DLQ replay is safe and drilled

- **Given** dead-lettered messages
- **When** the replay tool runs
- **Then** it re-publishes with the same `idempotency_key` and produces no duplicate writes
- **And** poison messages archive after three attempts with `error_code` retained

AC-4 • The findings change the config, not just the report

- **Given** the drill's results
- **When** they are reviewed
- **Then** every threshold or message the drill showed to be wrong is changed in config before the drill is considered complete

Technical notes

- AC-4 is what separates a drill from a ceremony. The `user_message` strings in particular will read badly under real conditions in a way they never do in review — that is precisely why §18.2 puts them in config rather than code.

Test cases

File	Case	Proves
tests/test_circuit_breakers.py	test_every_dependency_has_drilled_mode	AC-1
tests/test_circuit_breakers.py	test_recovery_drains_without_duplicates	AC-2
tests/test_kafka_roundtrip.py	test_dlq_replay_no_duplicate_writes	AC-3

18. Build Order — Fifteen Weeks, Fifteen Demos

The sequence below is optimised for one property: **something demonstrable every Friday**. Three cofounders shipping a feature a week cannot afford a four-week stretch where the answer to "what did we build" is "infrastructure."

The two spikes run first because each can force a topology change, and a topology change discovered in week 6 costs a sprint. A-04 — the Redis **databases 28** change — is in week 1 for the same reason: it is a platform-wide config change affecting every service on the instance, and nothing that touches Redis can land until it does.

Week	Stories	Friday demo
1	A-01, A-02, A-03, A-04	Both spikes answered with numbers; Redis DB 27 exists; events flowing to Kafka
2	A-05, A-06, B-01	Service responds on :8120 with a registry, guardrail chain and RBAC; session model migrated
3	B-02, B-03, B-04, B-05	Full data model with the provenance constraint proven at the database level
4	B-06, B-07, B-08, C-01	Session, consent and recording APIs working; prep chat reaches the agent
5	C-02, C-03, C-04, C-05	Research brief and a generated questionnaire covering all three workflows, approved in chat
6	D-01, D-02, D-03, E-01	Meeting scheduled in-app and synced to Google Calendar; onboarding entry point live
7	E-02, F-01, F-02, F-03	Consent recorded, meeting recorded, audio durable in GCS through a simulated network drop
8	F-04, F-05	Live transcription visible in the meeting view with speaker labels
9	F-06, G-01, G-02	Redaction working; STT killed mid-demo and the meeting carries on
10	G-03, G-04, G-05, G-06	The full live assist — green signals, follow-ups, ad-hoc capture, three-workflow coverage
11	H-01, H-02, H-03, H-04	A document photographed on camera, read, and flagged for retake while still on the table
12	I-01, I-02, I-03, J-01	Recordings library with summaries, key-moment seek and searchable transcripts
13	J-02, J-03, J-04, J-05	A processed meeting filling all five wizard pages with provenance on every value
14	J-06, K-01, K-02, K-03, K-04, K-05	End to end: meeting → review → confirm → wizard → PDF with meeting evidence
15	L-01...L-05, M-01...M-05, N-01...N-03	Prompt flywheel emitting candidates; drills passing; the NFR numbers on a dashboard

Week 15 is deliberately overloaded and should be read as a signal rather than a plan. The L, M and N epics total 34 points and will not fit one week at any honest velocity. Two options are available and the team should pick one before week 12 rather than discovering the problem in week 14: pull L-01 into week 8 alongside F-04 (it has no dependency on the live path and unblocks nothing else), and move M-01 to week 9 where the guardrails it audits are being written anyway; or extend to seventeen weeks and stop pretending.

The honest recommendation is the first. L-01 and M-01 both belong earlier than they are scheduled, and moving them turns week 15 from 34 points into 28, which is still tight but is a plan rather than a hope.

19. Traceability — Story to Design to Requirement

Every story above cites its design sections and requirement IDs in its meta line. This appendix inverts that index so a reviewer can start from a requirement and find the stories that satisfy it — which is the direction a requirements review actually runs.

Requirement area	Requirements	Stories
Preparation and research	FR-PREP-01...08	C-01, C-02, C-03, C-04, C-05, G-06
Scheduling	FR-CAL-01...03	D-01, D-02, D-03
Recording	FR-REC-01...03	F-01, F-02, F-03
Live assistance	FR-LIVE-01...09	F-04, F-05, F-06, G-01...G-06
Capture and OCR	FR-CAP-01...06	H-01, H-02, H-03, H-04
Recordings library	FR-LIB-01...03	I-01, I-02, I-03
Processing and auto-fill	FR-PROC-01...06	J-01...J-06
Review, wizard and PDF	FR-REV-01...05	K-01...K-05
Prompt optimization	FR-OPT-01...06	L-01...L-05
GDPR and consent	FR-GDPR-01...04	F-01, G-01, M-02, M-03
Security and RBAC	NFR-SEC-01...06	A-06, B-04, K-02, M-01, M-05
Performance	NFR-PERF-01...03	F-05, G-03, J-01, N-02
Reliability	NFR-REL-01...03	F-03, F-06, N-03
Compatibility	NFR-COMPAT	B-03, E-01, K-03, K-04, K-05
Operations and quality	NFR-OPS-01, NFR-QA-01	A-03, M-04, N-01

Two rows deserve a note. **NFR-COMPAT** appears across five stories in four different epics, which is correct and deliberate: the promise that the five-page wizard still works without a meeting is not a feature to be built once but a property to be defended at every surface that touches it. **FR-PREP-08** — collect for all three workflows, not just the wizard — traces to C-03 and G-06 rather than to a single card, because it is a generation requirement at prep time and a measurement requirement at meeting time.

20. Document History

Version	Date	Change
1.0	July 2026	Initial backlog. 58 stories, 167 points, epics A-N, 15-week build order. Acceptance criteria captured as a condensed semicolon-joined column.
2.0	25 July 2026	Full rewrite against Design Document v2.0. 67 stories, 181 points. Every card now carries Given/When/Then acceptance criteria with numbered AC - n blocks, technical notes anchored to the real §4.4 file paths, named test cases mapped to the §22 test files, and bidirectional dependency edges. Document-level Definition of Ready and Definition of Done added. Story identifiers renumbered so that the four IDs the design cites by name — A-02, A-04, B-05 and L-02 — land on exactly the work the design describes. Compound stories split: v1.0's A3 became A-04 and A-05; v1.0's B1 became B-01, B-02 and B-05. Traceability appendix added. Build order annotated with an explicit warning about week 15's overload and a recommended remedy.

What changed and why it matters. v1.0's acceptance criteria column was headed "Acceptance criteria (condensed)" and held fragments — B3's entire criteria read *"Endpoints per design §9.2 with RBAC (Editor+ create, Viewer read); state transitions validated server-side (illegal transition → 409); OpenAPI docs"*. That is a summary of an intention, not a test anyone can fail. Every card in this version states its conditions in a form an engineer can implement against and a reviewer can check, and names the test file that will prove it. The point total rose by 14 not because scope grew but because splitting the compound stories exposed work that was previously hidden inside an estimate — which is the same reason the week-15 overload is now visible rather than discovered in November.